



RAPPORT DE STAGE

Stage d'été – Juin 2026

Spécialité : Génie Informatique

INTITULÉ DU PROJET

**Développement d'un assistant intelligent multilingue basé
sur les LLM pour la gestion automatisée des tâches et des
événements**

Élaboré par : Rawen ZGARNI

Réalisé au sein de : 3LM Solutions

Encadrante entreprise : Mme Souhir SAIDI



Remerciements

Je remercie Mme Souhir Saidi pour son encadrement et ses conseils tout au long de ce stage.

Je remercie également l'équipe de 3LM Solutions pour son accueil et les conditions de travail qui m'ont permis de mener ce projet dans de bonnes conditions.

Enfin, je remercie l'École Nationale d'Ingénieurs de Carthage pour son accompagnement durant cette expérience.

Résumé

Ce projet porte sur la conception et la réalisation de la composante intelligente d'un assistant de productivité multilingue, capable d'interpréter des requêtes formulées en langage naturel – saisies au clavier ou à la voix – et de les transformer en actions de gestion de tâches et d'événements. L'objectif est de tirer parti des modèles de langage pour la compréhension, tout en conservant un contrôle déterministe sur l'exécution des actions.

La solution repose sur une architecture hybride : le modèle de langage identifie l'intention et extrait les entités, tandis que le backend assure la validation, la normalisation, le routage et l'exécution via des connecteurs Todo et Agenda, après confirmation de l'utilisateur. L'entrée vocale est transcrite par un modèle Whisper accédé via l'API Groq puis injectée dans le même pipeline que le texte ; les réponses peuvent être lues par synthèse vocale. Des mécanismes de retry et de fallback améliorent la résilience face aux erreurs des fournisseurs externes.

La qualité est évaluée à plusieurs niveaux : tests des connecteurs, tests d'extraction des entités, tests de généralisation, tests de fallback, validation contractuelle de l'intégration Speech-to-Text et évaluation sur un jeu de 65 cas. Ces contrôles sont intégrés à un pipeline GitHub Actions (intégration continue et contrôle de régression). Les résultats incluent 33 cas sur 33 en généralisation et 100 % d'exactitude des intentions sur l'évaluation locale ; l'extraction des entités (précision de 57,6 %), la latence et la dépendance aux services externes restent les principales limites.

Mots-clés : intelligence artificielle, modèles de langage, assistant conversationnel, classification des intentions, extraction d'entités, Speech-to-Text, API REST, résilience, fallback, tests automatisés, GitHub Actions, intégration continue.

Abstract

This project covers the design and implementation of the intelligent component of a multilingual productivity assistant that interprets natural-language requests – typed or spoken – and turns them into task and event management actions. The goal is to leverage large language models for understanding while keeping deterministic control over action execution.

The solution follows a hybrid architecture : the language model identifies the intent and extracts entities, while the backend handles validation, normalization, routing and execution through Todo and Agenda connectors, after user confirmation. Voice input is transcribed by a Whisper model accessed through the Groq API and then fed into the same pipeline as text ; replies can be read aloud through speech synthesis. Retry and fallback mechanisms improve resilience against external provider errors.

Quality is assessed at several levels : connector tests, entity extraction tests, intent generalization tests, fallback tests, a contract-level validation of the Speech-to-Text integration, and an evaluation on a 65-case dataset. These checks are wired into a GitHub Actions pipeline (continuous integration and regression control). Results include 33 out of 33 correct cases in generalization and 100% intent accuracy on the local evaluation ; entity extraction precision (57.6%), latency and dependency on external services remain the main limitations.

Keywords : artificial intelligence, large language models, conversational assistant, intent classification, entity extraction, Speech-to-Text, REST API, resilience, fallback, automated testing, GitHub Actions, continuous integration.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Liste des figures	ix
Liste des tableaux	x
Liste des abréviations	xii
Introduction générale	1
1 Contexte général et problématique	2
Introduction	2
1.1 Présentation de l'organisme d'accueil	2
1.1.1 Présentation de l'entreprise	2
1.1.2 Département et environnement d'accueil	2
1.1.3 Déroulement du stage	2
1.2 Contexte du projet	2
1.2.1 Assistants intelligents de productivité	2
1.2.2 Gestion traditionnelle des tâches et des événements	3
1.2.3 Apport des modèles de langage	3
1.2.4 Limites des solutions existantes	3
1.3 Problématique et solution proposée	3
1.3.1 Problématique	3
1.3.2 Objectifs du projet	3
1.3.3 Solution proposée	4
1.3.4 Périmètre fonctionnel et limites du projet	4
1.4 Méthodologie de conduite du projet	4
1.4.1 Méthodologie de développement adoptée	4
1.4.2 Organisation et suivi des tâches	5
1.4.3 Planification du projet : diagramme de Gantt	5
Conclusion	6
2 État de l'art et justification des choix technologiques	7
Introduction	7
2.1 Intelligence artificielle conversationnelle	7

2.1.1	Modèles de langage (LLM)	7
2.1.2	Compréhension du langage naturel et classification des intentions	7
2.1.3	Extraction des entités	7
2.1.4	Défis du traitement multilingue	8
2.1.5	Interaction vocale : transcription et synthèse	8
2.2	Architectures des assistants intelligents	8
2.3	Fiabilité et résilience des systèmes LLM	9
2.3.1	Limites des modèles de langage	9
2.3.2	Gestion des erreurs et réponses incertaines	9
2.3.3	Stratégies de retry et de fallback	9
2.3.4	Gestion des erreurs des fournisseurs de modèles	9
2.4	Intégration des services de productivité	9
2.4.1	API REST et principes d'intégration	9
2.4.2	Connecteurs pour les tâches et pour les événements	10
2.4.3	Authentification et sécurisation des échanges	10
2.5	Évaluation des systèmes conversationnels	10
2.5.1	Classification des intentions	10
2.5.2	Extraction des entités	10
2.5.3	Généralisation et robustesse	10
2.5.4	Évaluation des régressions	10
2.6	Synthèse des choix technologiques	11
	Conclusion	11
3	Analyse et spécification des besoins	12
	Introduction	12
3.1	Identification des acteurs	12
3.1.1	Utilisateur	12
3.1.2	Services externes de productivité	12
3.1.3	Fournisseurs de modèles de langage	12
3.2	Identification des besoins	13
3.2.1	Besoins fonctionnels	13
3.2.2	Besoins non fonctionnels	13
3.2.3	Contraintes techniques et limites du périmètre	13
3.3	Spécification fonctionnelle	14
3.3.1	Cas d'utilisation global	14
3.3.2	Gestion des tâches	14
3.3.3	Gestion des événements et de l'agenda	14
3.3.4	Interaction conversationnelle multilingue et vocale	15
3.3.5	Gestion des erreurs et des demandes non reconnues	15

3.4	Modélisation des interactions	15
3.4.1	Diagrammes de cas d'utilisation	15
3.4.2	Descriptions textuelles des cas d'utilisation	15
3.4.3	Diagrammes de séquence système	15
3.5	Planification du développement	17
3.5.1	Product Backlog	17
3.5.2	Découpage des fonctionnalités	20
3.5.3	Planification des itérations	20
	Conclusion	20
4	Conception et architecture de la solution	21
	Introduction	21
4.1	Architecture globale du système	21
4.1.1	Vue d'ensemble	21
4.1.2	Organisation des composants	22
4.1.3	Flux global de traitement d'une requête	22
4.2	Conception du pipeline conversationnel	22
4.2.1	Réception et normalisation des requêtes	22
4.2.2	Détection et classification des intentions	23
4.2.3	Extraction et normalisation des entités	23
4.2.4	Construction des actions de productivité	23
4.2.5	Gestion des réponses conversationnelles	23
4.3	Conception de la stratégie de résilience	24
4.3.1	Gestion des erreurs du modèle principal	24
4.3.2	Mécanisme de fallback	24
4.3.3	Routage déterministe des actions	24
4.3.4	Gestion des cas non reconnus	24
4.4	Conception des connecteurs API	25
4.4.1	Architecture des connecteurs	25
4.4.2	Contrats d'interface Todo et Agenda	25
4.4.3	Gestion des paramètres, des réponses et des erreurs HTTP	25
4.4.4	Séparation entre logique conversationnelle et intégration externe	25
4.5	Conception de la persistance	26
4.6	Conception de la stratégie d'évaluation	26
4.6.1	Organisation des jeux de données de test	26
4.6.2	Définition des métriques	26
4.6.3	Seuils d'acceptation et détection des régressions	27
	Conclusion	27
5	Réalisation, tests et validation	29

Introduction	29
5.1 Environnement de développement	29
5.1.1 Environnement matériel et logiciel	29
5.1.2 Technologies et bibliothèques utilisées	29
5.1.3 Organisation du dépôt et gestion des versions	29
5.2 Réalisation du pipeline conversationnel	30
5.2.1 Traitement des requêtes	30
5.2.2 Classification des intentions	30
5.2.3 Extraction des entités	30
5.2.4 Normalisation et résolution des paramètres d'action	31
5.3 Interface utilisateur	31
5.4 Réalisation de l'interaction vocale	32
5.4.1 Acquisition et traitement de l'entrée vocale	32
5.4.2 Conversion Speech-to-Text	32
5.4.3 Intégration du texte transcrit dans le pipeline conversationnel	33
5.4.4 Synthèse vocale des réponses	33
5.4.5 Validation de l'intégration vocale	33
5.4.6 Limites de l'interaction vocale	33
5.5 Réalisation des mécanismes de résilience	34
5.5.1 Gestion des erreurs des fournisseurs LLM	34
5.5.2 Implémentation du fallback	34
5.5.3 Routage déterministe des actions	34
5.5.4 Gestion des réponses non reconnues	34
5.6 Intégration des connecteurs de productivité	35
5.6.1 Connecteur Todo	35
5.6.2 Connecteur Agenda	35
5.6.3 Requetes HTTP, réponses et authentification	35
5.6.4 Validation des contrats d'intégration	35
5.7 Tests et validation fonctionnelle	36
5.7.1 Stratégie de test	36
5.7.2 Tests des connecteurs	36
5.7.3 Tests d'extraction des entités	36
5.7.4 Tests de généralisation des intentions	36
5.7.5 Tests des mécanismes de fallback	37
5.7.6 Tests des scénarios conversationnels	37
5.8 Évaluation quantitative du système	37
5.8.1 Évaluation sur le jeu de données de référence	37
5.8.2 Classification des intentions	37
5.8.3 Rappel et précision de l'extraction des entités	38

5.8.4	Analyse des erreurs et des cas limites	38
5.8.5	Résultats et interprétation	38
5.9	Intégration continue et contrôle qualité	38
5.9.1	Automatisation des tests avec GitHub Actions	39
5.9.2	Exécution des validations dans le pipeline CI	39
5.9.3	Mise en place du mécanisme de régression	39
5.9.4	Analyse des résultats d'intégration continue	39
5.10	Bilan et limites de la réalisation	40
5.10.1	Fonctionnalités réalisées	40
5.10.2	Limites de l'intégration externe	40
5.10.3	Limites liées à l'environnement de déploiement	41
Conclusion		41
Conclusion générale et perspectives		42
Bibliographie		43
Webographie		44
A Configuration des connecteurs API		45
B Exemples de requêtes et réponses HTTP		46
C Exemples du jeu de données d'évaluation		47
D Résultats détaillés des tests automatisés		48
E Configuration du pipeline GitHub Actions		49

Table des figures

1.1	Exemple de carte Kanban de suivi d'une tâche	5
1.2	Planification des principales activités à l'aide d'un diagramme de Gantt	6
3.1	Diagramme de cas d'utilisation global de l'assistant intelligent	16
3.2	Diagramme de séquence : création d'une tâche (connecteur API; en mode stub , un connecteur simulé remplace le connecteur HTTP)	18
3.3	Diagramme de séquence : création d'un événement (la date est extraite au format ISO 8601 par le modèle; la durée est demandée en minutes puis validée et normalisée)	19
4.1	Architecture globale de l'assistant intelligent de productivité	21
4.2	Logique de résilience face aux erreurs du fournisseur LLM (un succès à toute étape reprend le pipeline)	24
5.1	Écran de conversation de l'assistant	31
5.2	Proposition d'action soumise à confirmation	31
5.3	Flux de l'interaction vocale : entrée par transcription, sortie par synthèse	32

Liste des tableaux

1.1	Principales phases de la planification	5
2.1	Comparaison des architectures d’assistants	8
2.2	Synthèse des choix technologiques et architecturaux	11
3.1	Acteurs du système	12
3.2	Besoins fonctionnels	13
3.3	Besoins non fonctionnels	13
3.4	Descriptions synthétiques des cas d’utilisation	17
3.5	Product Backlog	17
3.6	Planification par itérations	20
4.1	Composants et responsabilités	22
4.2	Intentions prises en charge	23
4.3	Niveaux de la stratégie de résilience	25
4.4	Traitement général des réponses HTTP par les connecteurs	25
4.5	Métriques d’évaluation	27
4.6	Seuils de référence du regression gate	27
5.1	Environnement et technologies utilisés	29
5.2	Contrat de l’endpoint de transcription	32
5.3	Points vérifiés par la validation contractuelle du Speech-to-Text	33
5.4	Comportements implémentés selon le type d’erreur LLM	34
5.5	Catégories de tests	36
5.6	Scénarios conversationnels de référence	37
5.7	Évaluation locale du jeu de données (65 cas)	38
5.8	Jobs et validations du workflow GitHub Actions	39
5.9	Évaluation LLM du run CI #69 (exécution dégradée)	40
5.10	Regression gate lors de la validation CI considérée (comparaison aux seuils du tableau 4.6)	40
5.11	Les quatre niveaux de résultats d’évaluation	40
5.12	Fonctionnalités réalisées	41
A.1	Responsabilités et opérations des connecteurs	45
A.2	Éléments de configuration externalisés	45
B.1	Échange de transcription vocale	46
B.2	Échanges de traitement d’une requête et de confirmation	46

C.1	Exemples de requêtes par intention	47
D.1	Résultats des validations automatisées	48

Liste des abréviations

API	Application Programming Interface
CI / CD	Continuous Integration / Continuous Deployment
FN, FP, TP	Faux négatifs, faux positifs, vrais positifs
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model (grand modèle de langage)
NLP	Natural Language Processing
ORM	Object-Relational Mapping
REST	Representational State Transfer
STT	Speech-to-Text (transcription vocale)
TTS	Text-to-Speech (synthèse vocale)
UC	Use Case (cas d'utilisation)
UML	Unified Modeling Language
WER	Word Error Rate (taux d'erreur sur les mots)

Introduction générale

La transformation numérique a profondément modifié la manière dont les utilisateurs organisent leurs activités : la gestion des tâches et la planification des événements sont devenues des enjeux de productivité. Parallèlement, les progrès du traitement automatique du langage naturel (NLP) et des grands modèles de langage (LLM) permettent d'interagir avec les applications en langage naturel, par écrit comme à la voix.

Un assistant conversationnel utile ne se limite toutefois pas à générer du texte. Il doit comprendre l'intention de l'utilisateur, identifier les informations nécessaires à une action, interagir correctement avec les services concernés et rester fiable malgré les ambiguïtés du langage, les erreurs des modèles et les défaillances des services externes.

Ce rapport s'inscrit dans le projet « Multilingual AI Productivity Assistant », réalisé au sein de 3LM Solutions. Le travail porte sur la composante intelligente de l'assistant : interprétation des requêtes (texte ou voix), classification des intentions, extraction des entités, orientation des actions vers des connecteurs Todo et Agenda, mécanismes de retry et de fallback, ainsi qu'une validation reposant sur des tests automatisés, des jeux de données d'évaluation et un contrôle de régression. L'objectif est de transformer des requêtes en langage naturel en actions de productivité structurées, avec un accent sur la modularité, la robustesse et la testabilité.

Le rapport comporte cinq chapitres. Le chapitre 1 présente le contexte, l'organisme d'accueil, la problématique et la méthodologie. Le chapitre 2 dresse l'état de l'art et justifie les choix technologiques. Le chapitre 3 spécifie les besoins et modélise les interactions. Le chapitre 4 détaille la conception de la solution. Le chapitre 5 expose la réalisation, l'interaction vocale, les tests, l'évaluation quantitative, l'intégration continue et les limites. Une conclusion générale présente les contributions et les perspectives.

Chapitre 1

Contexte général et problématique

Introduction

Ce chapitre présente l'organisme d'accueil et le déroulement du stage, le contexte des assistants de productivité, la problématique et la solution proposée, puis la méthodologie de conduite du projet.

1.1 Présentation de l'organisme d'accueil

1.1.1 Présentation de l'entreprise

3LM Solutions, organisme d'accueil de ce stage d'été, évolue dans le domaine des solutions informatiques et du développement logiciel. Les travaux ont porté sur le projet Multilingual AI Productivity Assistant, dont l'objectif est de faciliter l'interaction entre les utilisateurs et les fonctionnalités de gestion des tâches et des événements à travers le langage naturel. Cette expérience a permis de découvrir les dimensions d'une solution logicielle intégrant l'intelligence artificielle : compréhension des requêtes, intégration de services et validation des fonctionnalités.

1.1.2 Département et environnement d'accueil

Le stage s'est déroulé à distance, au sein de l'environnement de développement de 3LM Solutions, sous l'encadrement de Mme Souhir Saidi. Cette organisation a exigé de l'autonomie et un suivi régulier de l'avancement. Les activités ont principalement concerné la composante intelligente de l'application : traitement des requêtes en langage naturel, identification des intentions et des entités, gestion des mécanismes de fallback et intégration de connecteurs de productivité.

1.1.3 Déroulement du stage

Le stage a duré un mois, au cours du mois de juin, en mode entièrement distant. Les travaux ont été répartis selon quatre axes : (i) prise en main du projet et de l'architecture existante ; (ii) développement des fonctionnalités (traitement conversationnel, intentions, entités) ; (iii) intégration et fiabilisation (connecteurs, fallback, gestion des erreurs) ; (iv) tests et validation automatisés. Le suivi des tâches est décrit en section 1.4.2.

1.2 Contexte du projet

1.2.1 Assistants intelligents de productivité

Les assistants intelligents de productivité facilitent l'organisation des activités quotidiennes (tâches, événements, informations personnelles). Contrairement aux applications traditionnelles, ils exploitent le traitement du langage naturel pour laisser l'utilisateur s'exprimer librement, par exemple pour créer une tâche ou déplacer un événement. L'assistant doit interpréter la requête, identifier l'action et les informations nécessaires, puis les transmettre aux services concernés.

1.2.2 Gestion traditionnelle des tâches et des événements

Les outils classiques reposent sur des interfaces graphiques et des opérations prédéfinies : la création d'une tâche ou d'un événement impose de renseigner des champs (titre, date, heure, description). Cette approche est structurée et prévisible, mais contraint l'utilisateur à s'adapter à l'interface, surtout lorsque plusieurs opérations s'enchaînent ou que l'information est exprimée naturellement. Un assistant conversationnel ajoute une couche d'interprétation entre l'utilisateur et ces services.

1.2.3 Apport des modèles de langage

Les LLM traitent des requêtes en langage naturel et en extraient des informations structurées. Dans le projet, le modèle intervient dans une chaîne comprenant la réception de la requête, la compréhension de l'intention, l'extraction des entités, la validation, la proposition d'une action et, après confirmation, son exécution par un connecteur. Cette architecture sépare la compréhension du langage de l'exécution des opérations métier.

1.2.4 Limites des solutions existantes

Malgré leurs progrès, les assistants fondés sur des LLM présentent plusieurs difficultés : interprétations incorrectes lorsque la requête est ambiguë, implicite ou incomplète ; dépendance à des services externes (disponibilité, erreurs réseau, limitations de débit, latence) ; difficulté d'évaluation, car il faut mesurer la capacité à reconnaître les intentions et à extraire les entités, pas seulement vérifier que l'application fonctionne. Ces contraintes justifient une architecture combinant validation des sorties du modèle, retry, fallback, connecteurs dédiés et évaluation automatisée.

1.3 Problématique et solution proposée

1.3.1 Problématique

Les outils de productivité classiques exigent une intervention manuelle structurée. L'intégration d'un LLM permet une interaction plus naturelle, mais une mauvaise interprétation peut entraîner une opération incorrecte, et la dépendance à des fournisseurs externes expose l'application aux erreurs réseau, aux limitations de débit et à la variabilité des réponses. La problématique s'énonce ainsi :

Comment concevoir un assistant intelligent multilingue capable de transformer des requêtes exprimées en langage naturel en actions structurées sur des services de productivité, tout en garantissant la fiabilité de l'exécution, la gestion des erreurs et l'évaluation automatisée du comportement du système ?

1.3.2 Objectifs du projet

L'objectif principal est de concevoir et d'intégrer une chaîne de traitement basée sur les LLM permettant d'interagir en langage naturel avec des fonctionnalités de productivité. Les objectifs détaillés sont :

- comprendre des requêtes formulées en langage naturel, dans plusieurs langues, et identifier leur intention ;
- extraire les entités nécessaires (titres, dates, heures, personnes, cibles) et les transformer en

- action structurée et contrôlable ;
- intégrer des connecteurs Todo et Agenda et introduire une confirmation avant l'exécution des actions sensibles ;
- offrir une modalité d'entrée vocale (transcription) et une restitution vocale des réponses ;
- assurer la robustesse des appels aux modèles (retry, fallback) et gérer les erreurs du modèle, du réseau et des services externes ;
- mettre en place un jeu de données d'évaluation et intégrer ces validations dans une chaîne d'intégration continue pour détecter les régressions.

1.3.3 Solution proposée

La solution est un assistant capable de convertir une requête en action de productivité structurée. La requête (saisie ou transcrite) est transmise au module d'interprétation basé sur un LLM, qui identifie l'intention et extrait les entités. Les informations sont validées et transformées en représentation structurée ; pour les actions modifiant des données, une proposition est présentée à l'utilisateur avant exécution. Le système sélectionne ensuite le connecteur correspondant, ce qui isole l'accès aux services externes de l'interprétation du langage. Selon l'erreur rencontrée, des tentatives limitées ou un basculement vers un modèle de secours sont appliqués. Enfin, une couche d'évaluation automatisée exécute des cas de test couvrant les intentions et l'extraction, avec des mécanismes de détection de régression. L'ensemble constitue une chaîne complète – compréhension, extraction structurée, orchestration, intégration d'API, fiabilité et évaluation – plutôt qu'un simple chatbot.

1.3.4 Périmètre fonctionnel et limites du projet

Le périmètre couvre la partie intelligence artificielle et intégration : création, modification et suppression de tâches et d'événements ; interactions conversationnelles (salutations, remerciements, demandes de capacités) ; interprétation, extraction, confirmation, communication avec les connecteurs, gestion des erreurs LLM ; entrée vocale (STT) et restitution vocale (TTS) ; validation par tests déterministes, généralisation, tests des connecteurs et évaluation automatisée dans GitHub Actions.

Restent hors périmètre : l'authentification globale de l'application, gérée séparément ; un déploiement continu complet (la chaîne GitHub Actions couvre l'intégration continue, la validation et l'évaluation) ; les environnements, identifiants et contraintes de disponibilité des services externes et des fournisseurs de modèles ; l'entraînement ou le fine-tuning d'un modèle.

1.4 Méthodologie de conduite du projet

1.4.1 Méthodologie de développement adoptée

Une approche itérative et incrémentale a été retenue, adaptée à un projet d'IA dont le comportement doit être vérifié sur plusieurs scénarios avant d'être jugé fiable. Le cycle suivi est : analyse → conception → développement → tests → validation → amélioration. Les fonctionnalités étaient implémentées sous forme de tâches distinctes, testées à chaque évolution importante pour détecter les régressions, avec une séparation constante entre compréhension par le LLM et exécution des actions. Les appels aux modèles ont été accompagnés progressivement de stratégies de retry et de fallback.

1.4.2 Organisation et suivi des tâches

Le suivi a reposé sur un tableau Kanban à trois colonnes : *To Do* (tâches identifiées, non commencées), *In Progress* (tâches en cours de développement ou de validation) et *Done* (tâches terminées après vérification). Chaque activité était représentée par une carte, déplacée selon son avancement, ce qui facilitait la priorisation et la visibilité sur l'état du projet dans le contexte du travail à distance. Le suivi était assuré sous la supervision de Mme Souhir Saidi, avec des échanges pour clarifier les besoins, discuter les choix techniques et valider les étapes.

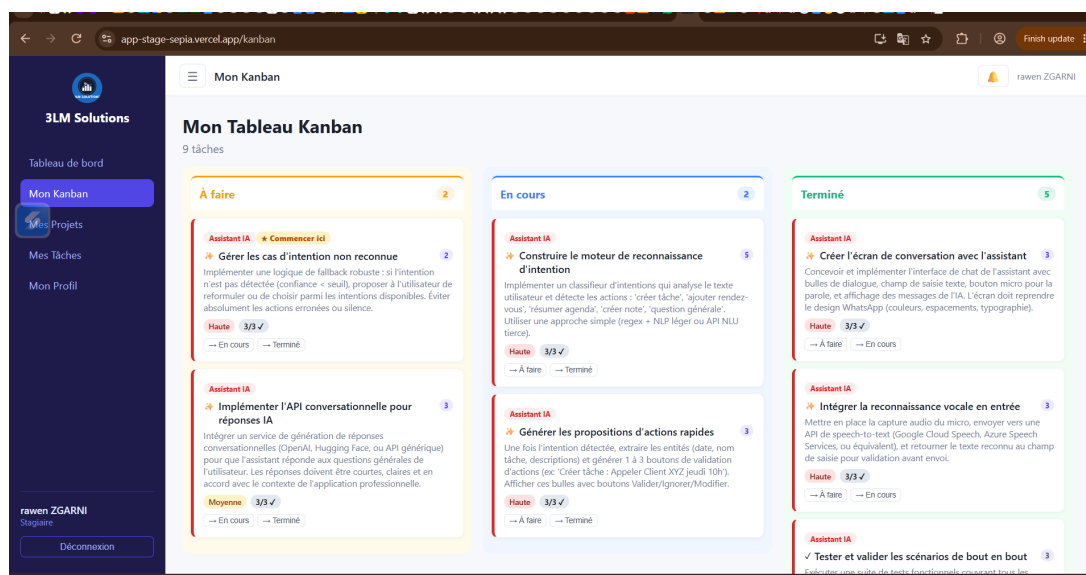


Figure 1.1 – Exemple de carte Kanban de suivi d'une tâche

1.4.3 Planification du projet : diagramme de Gantt

Le diagramme de Gantt complète le Kanban : le Kanban donne une vision opérationnelle de l'état des tâches, le Gantt une vision globale de leur répartition dans le temps. Le stage durant environ un mois, les activités ont été organisées en phases successives, tout en conservant de la flexibilité.

Table 1.1 – Principales phases de la planification

Phase	Principales activités
Prise en main et analyse	Compréhension de l'existant, analyse de l'architecture, identification des besoins
Conception	Définition du pipeline d'interprétation et des interfaces entre composants
Développement	Implémentation des fonctionnalités d'IA et du traitement des requêtes
Intégration	Mise en place des connecteurs Todo et Agenda
Fiabilisation	Retry, fallback, gestion des erreurs, scénarios d'échec
Tests et évaluation	Tests fonctionnels, généralisation, extraction d'entités, évaluation automatisée
CI et validation finale	Intégration des tests dans GitHub Actions, vérification globale
Documentation	Documentation technique et livrables du stage

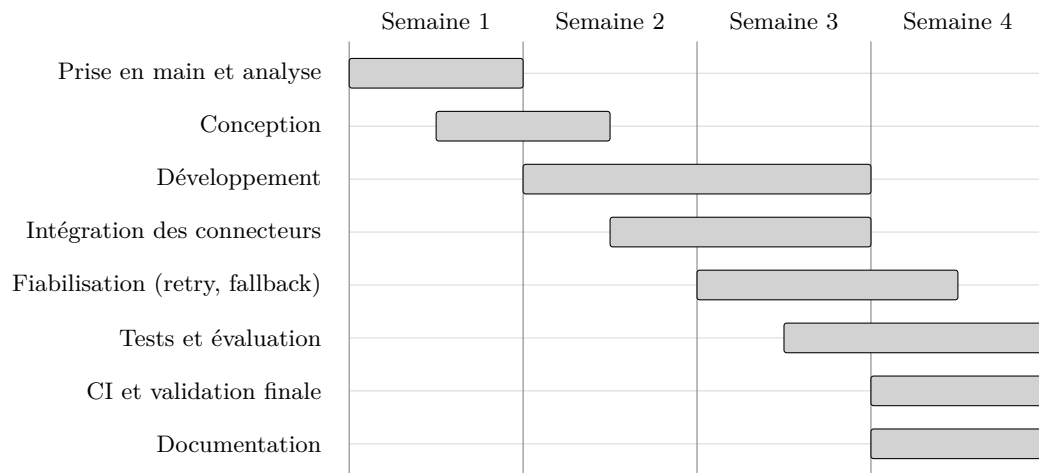


Figure 1.2 – Planification des principales activités à l’aide d’un diagramme de Gantt

Conclusion

Ce chapitre a présenté le cadre du stage, la problématique et la méthodologie. L’approche itérative, le suivi Kanban et la planification par phases ont structuré le développement en accordant une place centrale à la validation et à la fiabilité. Le chapitre suivant étudie les approches existantes et justifie les choix technologiques.

Chapitre 2

État de l’art et justification des choix technologiques

Introduction

Alors que les premiers assistants reposaient sur des règles et des arbres de décision, l’émergence des LLM permet d’interpréter des requêtes plus variées. Dans un assistant de productivité, le modèle ne génère pas seulement du texte : il doit transformer une demande libre (par exemple « Rappelle-moi de préparer le rapport demain matin ») en représentation structurée exploitable par l’application. Ce chapitre présente les concepts nécessaires, compare les architectures possibles, aborde la fiabilité des systèmes LLM, l’intégration des services et l’évaluation, puis synthétise les choix retenus.

2.1 Intelligence artificielle conversationnelle

Un assistant conversationnel peut être vu comme une chaîne : requête → compréhension → interprétation → extraction d’informations → décision → action ou réponse. Les systèmes modernes utilisent des modèles de langage pour généraliser leur compréhension à des formulations différentes.

2.1.1 Modèles de langage (LLM)

Les LLM sont entraînés sur de grandes quantités de texte et reposent généralement sur l’architecture Transformer et son mécanisme d’attention [7]. Ils génèrent des séquences de tokens à partir d’un contexte et peuvent comprendre une requête, identifier une intention, extraire des informations, résumer ou produire une sortie structurée destinée à une application – capacité particulièrement importante ici. Un LLM n’est cependant pas un composant déterministe : ses sorties varient selon le contexte, le modèle et les paramètres, et peuvent être incorrectes ou incomplètes. Son intégration exige donc des mécanismes de contrôle, de validation et de sécurisation, surtout lorsqu’une sortie peut modifier des données réelles.

2.1.2 Compréhension du langage naturel et classification des intentions

L’intention (*intent*) représente l’objectif de l’utilisateur. Des formulations différentes (« Ajoute une tâche pour préparer le rapport », « Rappelle-moi de préparer le rapport ») correspondent à une même catégorie fonctionnelle : la création de tâche. La classification associe une requête à une catégorie prédéfinie (création, modification, suppression de tâches et d’événements, mais aussi salutations, remerciements, demandes de capacités). Un LLM dépasse la correspondance exacte de mots-clés, mais une mauvaise classification peut déclencher une action incorrecte : elle doit être associée à des mécanismes de validation.

2.1.3 Extraction des entités

L’intention ne suffit pas pour exécuter une action : l’extraction d’entités identifie titre, date, heure, personne, lieu, période ou information à modifier. Pour « Planifie une réunion avec Karim

vendredi à 10 heures », l’intention est une création d’événement et les entités sont le participant, le jour et l’heure. L’intention détermine *ce que* l’utilisateur veut faire ; les entités fournissent les paramètres. Les expressions temporelles (« demain », « dans deux heures », « ce soir ») exigent une interprétation contextuelle, d’où la combinaison de l’interprétation du modèle et de vérifications applicatives.

2.1.4 Défis du traitement multilingue

Une même intention peut s’exprimer avec des structures, un vocabulaire et des conventions différents selon la langue. Les principaux défis sont la variabilité linguistique, l’ambiguïté contextuelle, les expressions temporelles propres à chaque langue, les variations lexicales (synonymes, abréviations, registre familier) et surtout l’évaluation de la généralisation sur des formulations non vues. L’approche multilingue vise une représentation des intentions et des entités indépendante de la langue, ce qui justifie une stratégie d’évaluation par formulations variées (section 4.6).

2.1.5 Interaction vocale : transcription et synthèse

La voix ajoute une modalité d’entrée et de sortie. La transcription (STT) convertit l’audio en texte ; des modèles comme Whisper, entraînés à grande échelle de façon faiblement supervisée, sont conçus pour la reconnaissance vocale multilingue [8]. La synthèse vocale (TTS) lit un texte à voix haute. Dans le projet, la transcription est déléguée à un service externe (Whisper via l’API Groq) et la synthèse à la bibliothèque Expo Speech côté frontend ; le texte transcrit réintègre ensuite le pipeline conversationnel standard (section 5.4).

2.2 Architectures des assistants intelligents

Le choix d’architecture dépend de la flexibilité recherchée, de la complexité des requêtes et du degré de contrôle nécessaire sur les actions exécutées – critique ici, puisque le système peut créer, modifier ou supprimer des données.

Table 2.1 – Comparaison des architectures d’assistants

	À base de règles	Fondée sur un LLM	Hybride (retenue)
Principe	Mots-clés et motifs explicites	Le LLM interprète et décide	LLM pour comprendre, logique applicative pour contrôler
Avantages	Déterministe, simple, contrôlable	Formulations variées, multilingue, contexte	Flexibilité et contrôle des opérations
Limites	Nombre de règles croissant, fragile aux variations, coûteux en multilingue	Sorties non déterministes, dépendance au fournisseur, risque si l’exécution est directe	Complexité d’orchestration, nécessite validation et tests

Les systèmes à base de règles offrent un contrôle important mais deviennent difficiles à maintenir dès que la diversité des requêtes augmente, notamment en contexte multilingue. Une architecture entièrement pilotée par un LLM est flexible mais introduit un risque lorsque sa sortie déclenche directement une opération sur les données.

L’architecture **hybride** a été retenue. Le LLM prend en charge l’interprétation (classification, extraction) ; la logique applicative assure les opérations déterministes (validation, normalisation, sélection du connecteur, gestion des erreurs, orchestration) ; une étape de confirmation constitue la frontière entre l’interprétation probabiliste et l’exécution déterministe ; les connecteurs encapsulent

l'accès aux API. Un changement de fournisseur LLM ou de service externe reste localisé au composant concerné. À cela s'ajoutent des mécanismes transverses de résilience et une couche d'évaluation automatisée intégrée à l'intégration continue.

2.3 Fiabilité et résilience des systèmes LLM

2.3.1 Limites des modèles de langage

Les principales limites à prendre en compte sont : la variabilité des sorties ; le risque d'interprétation incorrecte (intention erronée, entité inventée ou omise) ; les sorties structurées invalides ou incomplètes malgré un format demandé, qui imposent une validation applicative ; la dépendance au contexte fourni ; et les contraintes de disponibilité des API externes (limitations de débit, quotas, délais, erreurs temporaires).

2.3.2 Gestion des erreurs et réponses incertaines

On distingue les *erreurs d'interprétation* (mauvaise intention, entité incorrecte, informations insuffisantes) des *erreurs techniques* (réseau, délai, limitation de débit, erreur serveur), qui ne se traitent pas de la même manière. Face à une réponse insuffisamment fiable, une demande de clarification est préférable à l'exécution d'une action. Le traitement suit le principe : requête → interprétation → validation → décision (poursuivre, clarifier, réessayer, basculer ou échouer de façon contrôlée).

2.3.3 Stratégies de retry et de fallback

Le *retry* réessaie après une erreur temporaire ; il doit être borné pour ne pas aggraver la latence ou la surcharge, et certaines erreurs (authentification, configuration) ne doivent pas être réessayées. Le *fallback* utilise un modèle ou un service de secours lorsque le principal reste indisponible ; il ne masque pas toutes les erreurs et doit prévoir un état d'échec contrôlé si le principal et le secours échouent.

2.3.4 Gestion des erreurs des fournisseurs de modèles

Les erreurs fournisseur se classent en limitation de débit (429), erreurs serveur (5xx), erreurs réseau et délais d'attente, erreurs d'authentification ou de configuration, et échec du fallback ; chaque catégorie appelle un comportement distinct (retry, fallback ou arrêt contrôlé), évitant une stratégie uniforme qui répète toutes les requêtes. Le détail de la stratégie retenue est présenté en section 4.3.

2.4 Intégration des services de productivité

2.4.1 API REST et principes d'intégration

Les API REST permettent à des applications de communiquer par HTTP en opérant sur des ressources (GET, POST, PUT/PATCH, DELETE) avec des données généralement au format JSON. Un connecteur sert d'intermédiaire : il transforme les données internes en requêtes compatibles avec l'API puis interprète les réponses, en distinguant les codes de statut (succès, authentification, ressource inexistante, erreur serveur). Les données issues d'une phrase interprétée par un LLM doivent être validées avant transmission.

2.4.2 Connecteurs pour les tâches et pour les événements

Pour limiter le couplage avec une API donnée, le pipeline s’appuie sur une interface (assistant → interface → connecteur → API) : il demande une opération (créer, modifier, supprimer, consulter) sans connaître les détails de l’API. Le connecteur construit les requêtes HTTP, convertit les données, transmet l’authentification et traite les réponses et erreurs. Cette abstraction permet de remplacer un service, ou d’utiliser un connecteur simulé en test. Les événements ajoutent des contraintes temporelles (date, heures de début et de fin, participants) ; « demain matin » ou « vendredi prochain » doivent être normalisés avant envoi.

2.4.3 Authentification et sécurisation des échanges

L’accès aux API repose sur des en-têtes HTTP (clés API, jetons, OAuth 2.0). Les identifiants ne doivent pas figurer dans le code source : ils sont fournis par la configuration de l’environnement et centralisés dans la couche des connecteurs. Les échanges utilisent HTTPS, les paramètres issus du LLM sont validés avant d’être transmis, et les contraintes de limitation de débit des services doivent être respectées.

2.5 Évaluation des systèmes conversationnels

Un assistant fondé sur un LLM ne peut être jugé fiable sur quelques exemples : une modification du modèle, du prompt ou de l’orchestration peut dégrader certaines fonctionnalités sans erreur technique visible. L’évaluation porte ici sur les éléments structurés nécessaires à l’action : intention, entités, généralisation et absence de régression.

2.5.1 Classification des intentions

Elle compare, sur des requêtes annotées, l’intention prédite à l’intention attendue (mesure d’exactitude). Le projet distingue les tests de classification classiques des tests de généralisation, car l’évaluation sur des formulations proches de celles du développement est insuffisante.

2.5.2 Extraction des entités

Elle compare les entités extraites aux entités attendues selon le rappel (entités attendues retrouvées), la précision (entités extraites réellement pertinentes) et l’exactitude des valeurs. Une extraction trop restrictive omet un paramètre nécessaire ; une extraction trop permissive ajoute des informations non demandées. Elle est évaluée indépendamment de l’intention, ce qui permet de localiser l’origine d’un problème. Les définitions formelles sont données en section 4.6.2.

2.5.3 Généralisation et robustesse

Les tests de généralisation utilisent des formulations différentes de celles de référence pour des intentions connues, afin de vérifier que le système ne dépend pas d’une correspondance lexicale directe. La robustesse concerne aussi les formulations indirectes, les informations partielles et les variations linguistiques.

2.5.4 Évaluation des régressions

Le projet compare les résultats courants à un niveau de référence (*baseline*) défini dans un fichier dédié (section 4.6.3). Une évaluation utilisant un LLM externe peut être perturbée par des limitations de débit ou des indisponibilités : une baisse apparente des résultats n’est alors pas

nécessairement une régression du code. Le pipeline distingue donc les erreurs liées au fournisseur des résultats utilisés pour le contrôle de régression.

2.6 Synthèse des choix technologiques

Table 2.2 – Synthèse des choix technologiques et architecturaux

Besoin	Choix retenu	Justification
Compréhension du langage	LLM	Gestion flexible des formulations et des langues
Actions et paramètres	Classification des intentions, extraction structurée des entités	Transformer la requête en action exploitable
Architecture	Approche hybride	Flexibilité du LLM, contrôle par la logique applicative
Exécution	Connecteurs de services	Séparation entre intelligence conversationnelle et API externes
Modalité vocale	STT (Whisper via Groq), TTS (Expo Speech)	Même pipeline pour voix et texte
Erreurs LLM	Retry borné et fallback	Résilience face aux erreurs temporaires
Validation	Tests automatisés, seuils de référence, CI	Vérification reproductible, détection des régressions

Conclusion

L’étude a conduit à une architecture hybride : le LLM comprend, la logique déterministe valide, route et exécute, des connecteurs modulaires isolent les services externes, la résilience maîtrise la dépendance aux fournisseurs et une évaluation automatisée détecte les régressions. Ces choix fondent la spécification, la conception et la réalisation présentées ensuite.

Chapitre 3

Analyse et spécification des besoins

Introduction

Ce chapitre traduit les objectifs du projet en exigences fonctionnelles et non fonctionnelles. Le système se situe à l'intersection de plusieurs composants – l'utilisateur, le modèle de langage, la logique applicative et les services externes – dont il faut préciser les rôles avant de modéliser leurs interactions et de planifier le développement.

3.1 Identification des acteurs

Table 3.1 – Acteurs du système

Acteur	Rôle principal	Interaction avec le système
Utilisateur	Formuler des demandes (texte ou voix) et contrôler les actions proposées	Envoie des requêtes, confirme ou refuse les actions
Services externes de productivité	Fournir la gestion des tâches et des événements	Reçoivent les opérations, retournent leurs résultats
Fournisseurs de modèles	Interpréter les requêtes (LLM) et transcrire la voix (STT)	Retournent intention et entités, ou texte transcrit

3.1.1 Utilisateur

Acteur principal, l'utilisateur s'exprime en langage naturel sans syntaxe imposée, dans plusieurs langues, au clavier ou à la voix. Il peut demander la création, la modification ou la suppression d'une tâche ou d'un événement, ou avoir un échange conversationnel (salutation, remerciement, demande de capacités). Pour les actions modifiant des données, il confirme ou refuse la proposition, ce qui conserve un contrôle humain. Le parcours est : requête → interprétation → proposition → confirmation → exécution → résultat. Il doit recevoir un retour compréhensible lorsque la demande est ambiguë, incomplète ou impossible.

3.1.2 Services externes de productivité

Ces services exécutent les actions validées sur leurs propres ressources ; ils ne participent pas à l'interprétation. L'accès passe par des connecteurs qui construisent les requêtes HTTP, transmettent les paramètres et l'authentification, convertissent les données et gèrent les réponses et erreurs. Des connecteurs simulés peuvent être utilisés en test pour reproduire des situations de succès ou d'erreur.

3.1.3 Fournisseurs de modèles de langage

Le fournisseur LLM transforme une expression non structurée en intention et entités, soumises ensuite à validation et orchestration. Il n'est pas considéré comme toujours disponible ni déterministe : il peut renvoyer une réponse invalide, imposer une limitation de débit ou être indisponible, d'où les mécanismes de validation, de retry et de fallback. Un service de transcription vocale joue un rôle analogue en amont du pipeline pour l'entrée vocale.

3.2 Identification des besoins

3.2.1 Besoins fonctionnels

Table 3.2 – Besoins fonctionnels

ID	Besoin	Description
BF1	Compréhension des requêtes	Analyser une requête libre, sans syntaxe imposée, et en déterminer le sens fonctionnel
BF2	Identification de l'intention	Distinguer les opérations sur tâches et événements (création, modification, suppression) et reconnaître les intentions conversationnelles
BF3	Extraction des entités	Extraire titre, date, heure, durée, cible à modifier ou supprimer, sous forme structurée
BF4	Validation et normalisation	Vérifier la cohérence de l'intention et des paramètres, détecter les informations manquantes ou invalides
BF5	Proposition et confirmation	Construire l'action avant exécution ; aucune transmission au service en cas de refus
BF6	Exécution via connecteurs	Transmettre l'opération validée au connecteur Todo ou Agenda et retourner le résultat ou l'erreur
BF7	Gestion des erreurs	Produire un comportement contrôlé en cas de requête incomplète, de donnée invalide, d'erreur d'API ou de fournisseur
BF8	Retry et fallback	Réessayer de façon limitée les erreurs temporaires ; basculer vers un modèle de secours lorsque c'est pertinent
BF9	Multilinguisme	Conserver une interprétation cohérente malgré les variations de vocabulaire, de syntaxe et de langue
BF10	Interaction vocale	Permettre la saisie vocale (enregistrement, transcription) et la lecture vocale des réponses, avec possibilité de la désactiver
BF11	Évaluation automatisée	Vérifier intentions, entités, généralisation, connecteurs, erreurs et régressions de façon reproductible en CI

3.2.2 Besoins non fonctionnels

Table 3.3 – Besoins non fonctionnels

Propriété	Exigence
Fiabilité et robustesse	Comportement cohérent et contrôlé face aux requêtes ambiguës ou aux erreurs externes ; une erreur ne doit pas provoquer l'exécution d'une action incorrecte ; tolérance aux variations de formulation
Disponibilité et résilience	Limiter l'impact des indisponibilités temporaires (retry, fallback) avec échec contrôlé lorsque la récupération est impossible
Performance	Temps de traitement compatible avec une interaction conversationnelle, en tenant compte de la latence propre aux LLM et aux API externes
Maintenabilité et extensibilité	Séparation interprétation / validation / orchestration / connecteurs ; ajout de connecteurs, d'intentions ou de scénarios sans réécriture du pipeline
Testabilité et traçabilité	Composants testables indépendamment (connecteurs simulés) ; conservation des résultats d'évaluation sous forme d'artefacts CI
Sécurité	Clés d'API et paramètres d'accès fournis par la configuration d'environnement, hors du code source ; validation des sorties du modèle avant usage

3.2.3 Contraintes techniques et limites du périmètre

- **Dépendances externes** : fournisseurs LLM et de transcription, et API de productivité (disponibilité, quotas, latence, évolution des contrats, authentification) ne sont pas maîtrisés par l'application ; les évaluations avec fournisseur réel doivent distinguer erreurs système et

erreurs fournisseur.

- **Variabilité des modèles** : deux requêtes proches peuvent produire des sorties différentes ; elles sont donc validées avant exécution.
- **Hors périmètre** : authentification globale de l'application ; déploiement continu vers la production (le pipeline couvre l'intégration continue et l'évaluation) ; fine-tuning de modèle ; plateforme universelle intégrant tous les services de productivité.
- **Données d'évaluation limitées** : les jeux de données ne représentent pas toutes les formulations d'utilisateurs réels ; les résultats sont des indicateurs de qualité dans le périmètre défini, non une garantie générale.

3.3 Spécification fonctionnelle

Le fonctionnement général est la chaîne : utilisateur → requête naturelle → interprétation LLM → intention et entités → validation → proposition → confirmation → connecteur → service externe → résultat. Elle sépare la compréhension de la demande de son exécution.

3.3.1 Cas d'utilisation global

L'utilisateur soumet une demande, consulte la proposition, la confirme ou la refuse, gère tâches et événements, et converse avec l'assistant ; le fournisseur LLM interprète la requête ; le service de productivité exécute l'opération ; le système traite les erreurs. Le scénario nominal est : (1) l'utilisateur formule une requête ; (2) elle est transmise à l'interprétation ; (3) intention et entités sont identifiées ; (4) les informations sont validées et normalisées ; (5) une action structurée est construite ; (6) elle est présentée pour confirmation ; (7) après confirmation, le connecteur est appelé ; (8) le service exécute l'opération ; (9) le résultat est présenté. En cas d'erreur ou d'information insuffisante, le flux est interrompu pour éviter toute opération incorrecte.

3.3.2 Gestion des tâches

Trois opérations sont prises en charge. La *création* extrait le titre et les paramètres utiles, valide, propose, puis passe par le connecteur Todo ; un paramètre indispensable manquant ou incohérent empêche l'exécution directe. La *modification* suppose d'identifier la tâche cible et la nouvelle information, en évitant de modifier une ressource incorrecte si plusieurs éléments correspondent. La *suppression* exige une identification suffisamment fiable de la cible et une confirmation. Le résultat distingue au minimum succès et échec, sans jamais présenter comme réussie une opération que le service n'a pas confirmée.

3.3.3 Gestion des événements et de l'agenda

La création, la modification et la suppression d'événements suivent le même principe, avec des contraintes temporelles : les expressions comme « vendredi après-midi » ou « demain à 10 heures » sont converties en date et heure exploitables (le modèle les exprime au format ISO 8601) avant l'appel au connecteur Agenda, et les informations ambiguës sont traitées dans le flux de validation avant exécution. Déplacer un rendez-vous implique d'identifier l'événement concerné et sa nouvelle date ou heure. Au niveau de la confirmation, l'implémentation prévoit également une valeur par défaut pour certains paramètres temporels absents ; ce comportement constitue un point d'amélioration identifié pour renforcer encore la politique « aucune valeur métier implicite ».

3.3.4 Interaction conversationnelle multilingue et vocale

La capacité multilingue ne consiste pas à traduire : elle vise à conserver la même structure fonctionnelle pour des intentions équivalentes exprimées dans d'autres langues ou formulations. Les intentions purement conversationnelles suivent le même pipeline sans déclencher de connecteur. L'utilisateur peut dicter sa requête : l'audio est transcrit, puis le texte obtenu suit exactement les mêmes étapes que la saisie clavier (section 5.4.3). Les réponses de l'assistant peuvent être lues à voix haute, avec un contrôle d'activation.

3.3.5 Gestion des erreurs et des demandes non reconnues

Une requête peut être comprise et exploitable, comprise mais incomplète, ambiguë, associée à une intention non prise en charge, ou impossible à traiter pour une raison technique. Le système ne doit pas inventer de valeur manquante : il identifie l'information absente et demande une précision lorsque le flux le permet. Une requête non reconnue reçoit une réponse contrôlée, sans appel de connecteur. Les erreurs techniques sont traitées séparément des erreurs d'interprétation : retry limité, puis fallback, puis échec contrôlé avec information de l'utilisateur.

3.4 Modélisation des interactions

3.4.1 Diagrammes de cas d'utilisation

Trois acteurs figurent dans le diagramme global : l'utilisateur (soumet une requête, crée, modifie ou supprime tâches et événements, confirme ou refuse, converse, reçoit le résultat ou une réponse pour demande non reconnue, utilise la voix), le fournisseur LLM (interprétation) et le service de productivité (exécution). Les cas liés aux opérations de productivité se spécialisent en gestion des tâches et gestion des événements. Les mécanismes internes (parsing, validation de schéma, gestion fine des erreurs) sont détaillés dans les diagrammes de séquence.

3.4.2 Descriptions textuelles des cas d'utilisation

3.4.3 Diagrammes de séquence système

Le scénario général est le suivant : l'utilisateur saisit ou dicte une requête ; le frontend l'envoie au backend (`POST /assistant/message`) ; pour les requêtes nécessitant une interprétation par le modèle, le backend appelle le fournisseur LLM, qui retourne l'intention et les entités ; le backend normalise les entités (`normalizeActionEntities`), construit une proposition d'action (`proposedAction`) et la renvoie au frontend, qui l'affiche pour confirmation. Certaines requêtes de modification de titre de tâche peuvent toutefois être traitées par le routage déterministe avant l'appel au LLM. Après validation par l'utilisateur, le frontend appelle `POST /assistant/confirm-action` ; le backend transmet l'opération au connecteur Todo ou Agenda, met à jour l'interaction en base et retourne le résultat. Le LLM ne communique jamais directement avec le service de productivité. La validation et l'orchestration de l'exécution sont portées par ces deux routes du backend, en coordination avec le composant d'interface de proposition d'action ; il n'existe pas de service d'orchestration distinct.

La *création d'un événement* suit la même séquence ; la date de l'événement est extraite par le modèle au format ISO 8601 et la durée est demandée par le schéma de classification directement en minutes, puis validée et normalisée par `normalizeActionEntities`. Aucune couche de normalisation temporelle indépendante n'est présente dans l'implémentation. Dans le

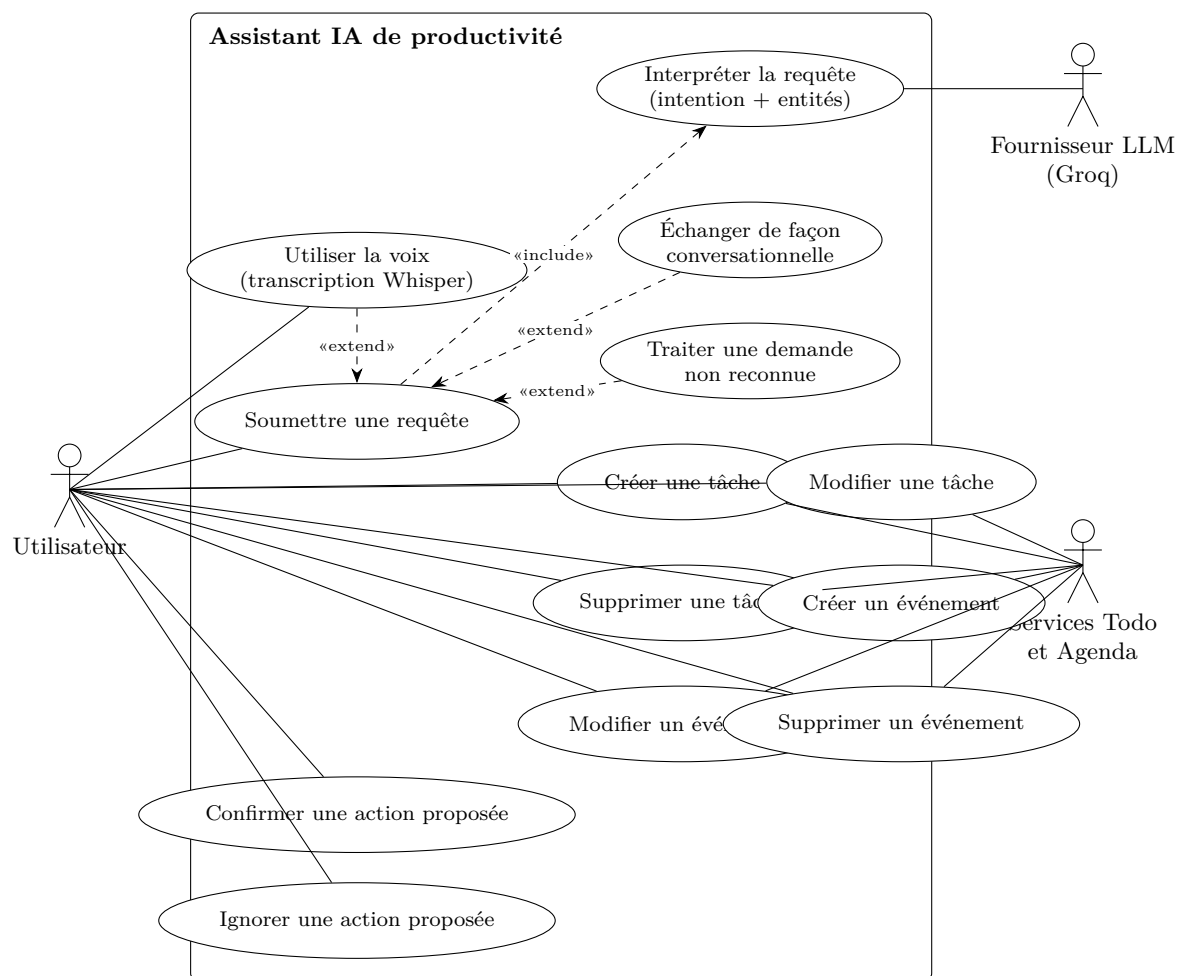
**Figure 3.1** – Diagramme de cas d'utilisation global de l'assistant intelligent

Table 3.4 – Descriptions synthétiques des cas d'utilisation

ID	Nom	Précondition	Scénario nominal / alternatives
UC-01	Soumettre une requête	Assistant disponible	Saisie ou dictée de la requête, préparation, transmission à l'interprétation, récupération d'une représentation structurée. <i>Alternatives</i> : requête vide ou invalide, service d'interprétation indisponible, intention non reconnue.
UC-02	Créer une tâche	La requête contient assez d'informations	Intention create_task , extraction des entités, validation, proposition, confirmation, requête du connecteur Todo, création, résultat. <i>Alternatives</i> : titre absent, données invalides, refus, erreur du service, retry sur erreur temporaire.
UC-03	Modifier une tâche	Tâche cible identifiable	Intention modify_task , extraction de la cible et des nouvelles valeurs, validation, proposition, confirmation, appel du connecteur, résultat. <i>Alternatives</i> : plusieurs ressources correspondent, cible introuvable, refus, erreur du service.
UC-04	Supprimer une tâche	Tâche cible identifiable	Intention delete_task , identification de la cible, validation, proposition, confirmation, appel du connecteur, résultat. <i>Alternatives</i> : cible introuvable, refus, erreur du service.
UC-05	Créer un événement	Informations nécessaires disponibles	Intention create_event , extraction de la date et de l'heure au format ISO 8601 et de la durée en minutes, validation, proposition, confirmation, appel du connecteur Agenda, résultat. <i>Alternatives</i> : information ambiguë ou absente, refus, service indisponible.
UC-06	Modifier un événement	Événement cible identifiable	Intention modify_event , extraction de la cible et des nouvelles valeurs, validation, proposition, confirmation, appel du connecteur Agenda, résultat. <i>Alternatives</i> : plusieurs ressources correspondent, cible introuvable, refus, erreur du service.
UC-07	Supprimer un événement	Événement cible identifiable	Intention delete_event , identification de la cible, validation, proposition, confirmation, appel du connecteur Agenda, résultat. <i>Alternatives</i> : cible introuvable, refus, erreur du service.
UC-08	Traiter une demande non reconnue	Requête reçue	Intention classée unrecognized , aucun connecteur appelé, réponse contrôlée à l'utilisateur ; une demande non reconnue n'est jamais transformée en action.
UC-09	Utiliser la voix	Permission micro accordée	Enregistrement, transcription, injection du texte dans le pipeline (UC-01), lecture vocale de la réponse si activée. <i>Alternatives</i> : audio absent, échec de transcription.

scénario d'*erreur*, une erreur temporaire du LLM principal déclenche une nouvelle tentative, puis, si elle persiste, un appel au modèle de secours ; si les deux échouent, le traitement se termine de façon contrôlée avec information de l'utilisateur (figure 4.2).

3.5 Planification du développement

3.5.1 Product Backlog

Table 3.5 – Product Backlog

ID	Élément	Prio.	ID	Élément	Prio.
B01	Compréhension des requêtes	Haute	B11	Requêtes non reconnues	Moy.
B02	Extraction des entités	Haute	B12	Support multilingue	Moy.
B03	Gestion des tâches	Haute	B13	Tests des connecteurs	Haute
B04	Gestion des événements	Haute	B14	Tests de généralisation	Haute
B05	Validation des actions	Haute	B15	Évaluation du modèle	Haute
B06	Proposition et confirmation	Haute	B16	Contrôle des régressions	Haute
B07	Connecteur Todo	Haute	B17	Validation frontend	Moy.
B08	Connecteur Agenda	Haute	B18	Intégration CI	Haute
B09	Gestion des erreurs	Haute	B19	Interaction vocale (STT/TTS)	Moy.
B10	Retry et fallback	Haute			

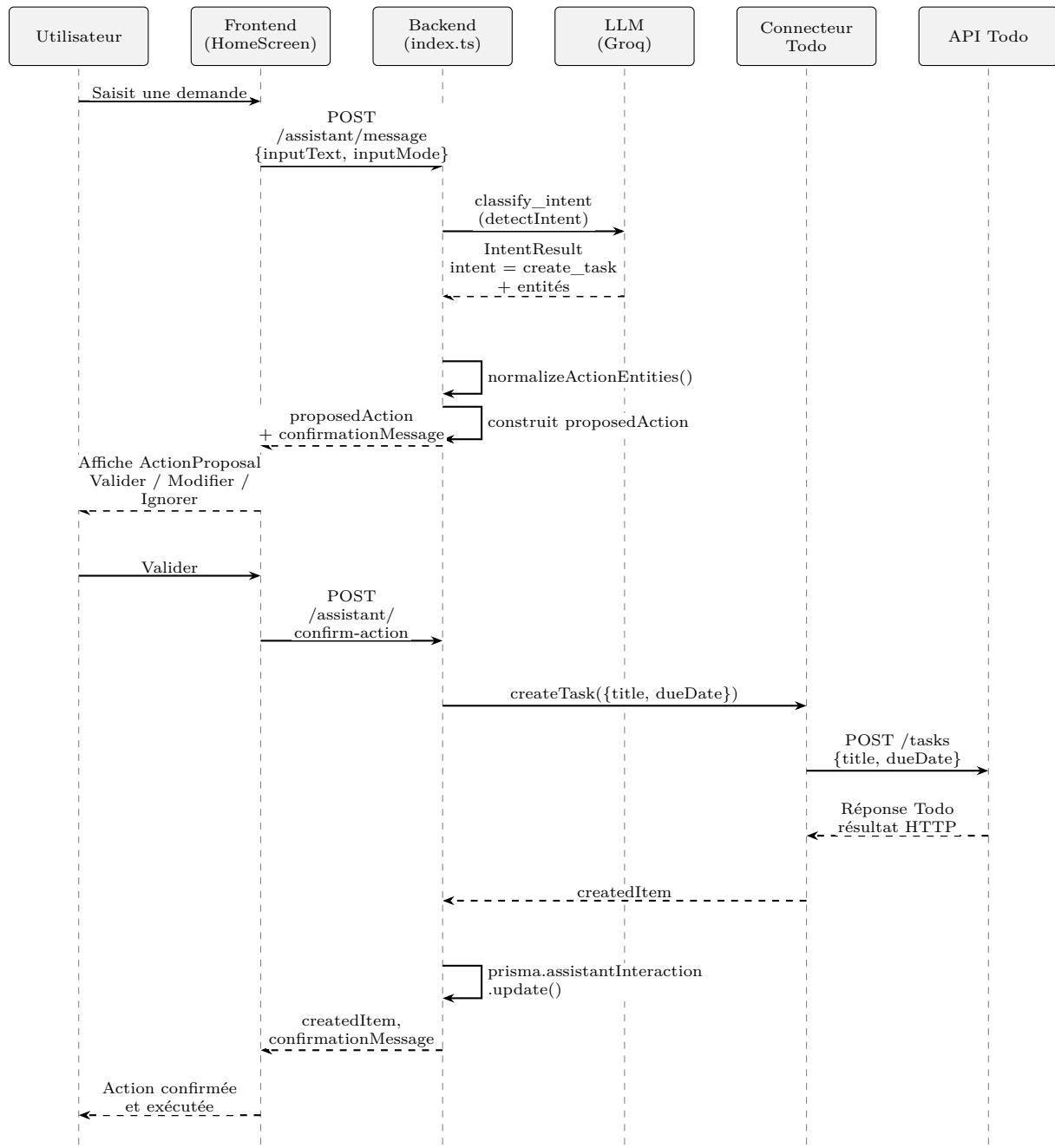


Figure 3.2 – Diagramme de séquence : création d’une tâche (connecteur API; en mode `stub`, un connecteur simulé remplace le connecteur HTTP)

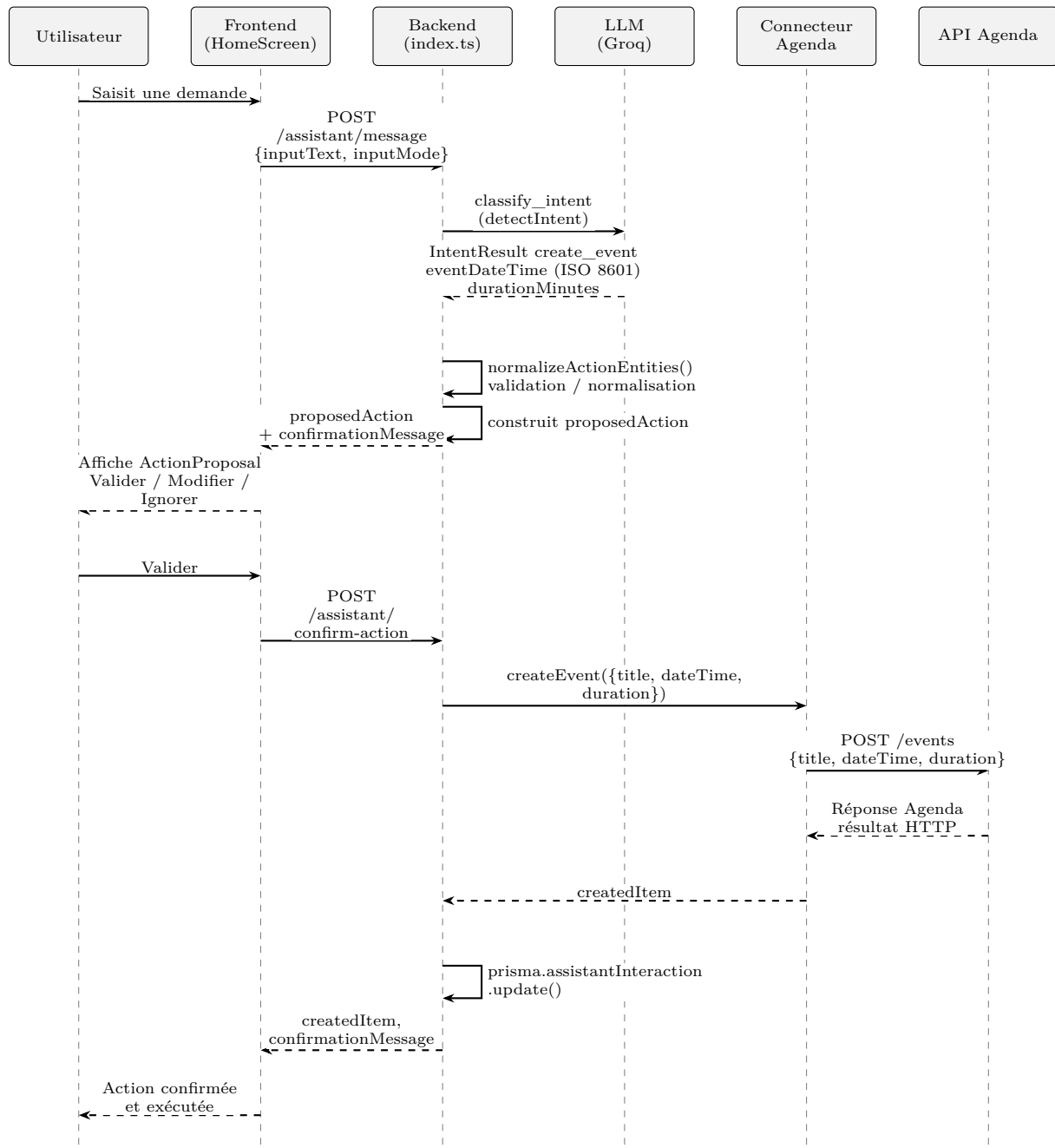


Figure 3.3 – Diagramme de séquence : création d'un événement (la date est extraite au format ISO 8601 par le modèle ; la durée est demandée en minutes puis validée et normalisée)

3.5.2 Découpage des fonctionnalités

Les fonctionnalités sont regroupées en six blocs : (1) compréhension du langage (intention, entités) ; (2) gestion fonctionnelle des tâches et événements ; (3) validation et orchestration, qui évite de confier au modèle l'exécution d'une opération externe ; (4) connecteurs d'intégration ; (5) résilience (erreurs fournisseur, retry, fallback) ; (6) validation et évaluation (tests déterministes, connecteurs, intentions et entités, généralisation, régressions). L'interaction vocale s'insère en amont (entrée) et en aval (sortie) du premier bloc sans modifier les suivants.

3.5.3 Planification des itérations

Table 3.6 – Planification par itérations

Période	Itération	Principaux travaux
Semaine 1	Analyse et compréhension	Analyse du besoin, étude de l'architecture existante, identification des intentions et entités, contrats fonctionnels
Semaine 2	Fonctionnalités principales	Compréhension des requêtes, extraction des entités, gestion des tâches et événements, premières intégrations de connecteurs
Semaine 3	Intégration et robustesse	Finalisation des connecteurs Todo et Agenda, validation des actions, gestion des erreurs, retry et fallback
Semaine 4	Évaluation et validation	Tests fonctionnels et techniques, généralisation, évaluation du modèle, contrôle des régressions, validation frontend, intégration CI

Ces itérations ne sont pas strictement séquentielles : certaines validations (tests des connecteurs, par exemple) ont accompagné l'implémentation pour détecter tôt les erreurs d'intégration.

Conclusion

L'analyse a défini les acteurs, les besoins fonctionnels et non fonctionnels, les contraintes et le périmètre, puis modélisé les principaux scénarios et planifié le développement. Elle prépare la conception de l'architecture présentée au chapitre suivant.

Chapitre 4

Conception et architecture de la solution

Introduction

Ce chapitre présente l'architecture technique retenue. La conception combine l'interprétation par le LLM avec des mécanismes déterministes de validation, d'orchestration et d'intégration : le modèle comprend et extrait, la logique applicative valide et exécute. L'architecture intègre des connecteurs dédiés, des mécanismes de retry et de fallback, une modalité d'entrée vocale et une stratégie d'évaluation.

4.1 Architecture globale du système

4.1.1 Vue d'ensemble

L'architecture est modulaire et organisée en niveaux : interface d'interaction, traitement intelligent, logique applicative, connecteurs et services externes. L'utilisateur formule une requête (texte ou voix) depuis l'application ; le backend la reçoit et coordonne le traitement. Le LLM identifie l'intention et extrait les entités, mais sa sortie n'est jamais une instruction d'exécution : la couche de validation et d'orchestration vérifie la cohérence, normalise et détermine l'action. Pour les actions modifiant une ressource, une proposition est soumise à confirmation. L'orchestrateur sélectionne ensuite le connecteur Todo ou Agenda, qui communique avec l'API externe. Le résultat remonte jusqu'à l'utilisateur ; en cas d'erreur, les mécanismes de résilience s'appliquent selon sa nature.

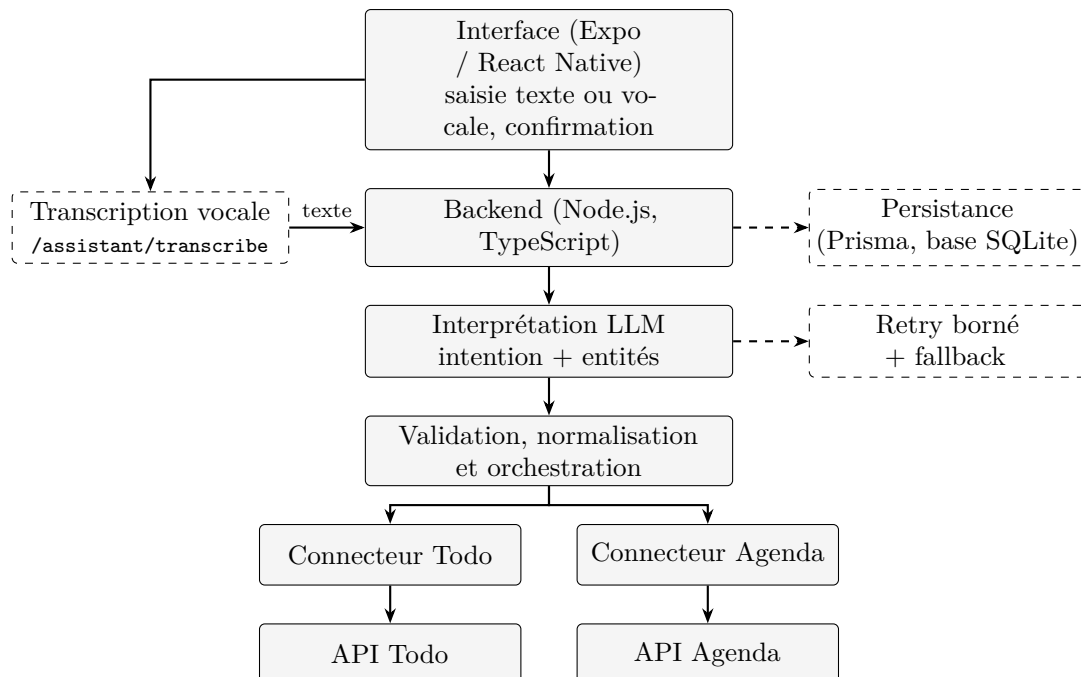


Figure 4.1 – Architecture globale de l'assistant intelligent de productivité

La boîte « Validation, normalisation et orchestration » représente une couche *lo-*

gique : dans l'implémentation, elle est portée par les routes `/assistant/message` et `/assistant/confirm-action` du backend, par la fonction `normalizeActionEntities` et par le composant d'interface de proposition d'action, et non par un service distinct.

4.1.2 Organisation des composants

Table 4.1 – Composants et responsabilités

Composant	Responsabilité principale
Interface utilisateur	Saisie (texte, voix), affichage des propositions, résultats et historique ; ne porte pas l'interprétation
Backend / API	Réception des requêtes et coordination du traitement
Transcription vocale	Conversion de l'audio en texte avant le pipeline conversationnel
Service d'interprétation LLM	Intention et entités à partir du langage naturel
Validation et orchestration	Vérification et normalisation, détermination de l'action, confirmation, sélection du connecteur
Connecteurs Todo et Agenda	Adaptation des opérations aux contrats des API externes (endpoints, méthodes, authentification, formats, réponses)
Retry et fallback	Gestion des erreurs temporaires des fournisseurs de modèles
Couche d'évaluation	Mesure du comportement et détection des régressions, intégrée à la CI

L'usage du LLM est volontairement limité à l'interprétation ; l'exécution n'est jamais confiée au modèle. Cette séparation est l'élément central de l'architecture hybride : le modèle propose, la logique déterministe contrôle.

4.1.3 Flux global de traitement d'une requête

Le traitement suit huit étapes : (1) *réception* par le backend, sans opération externe ; (2) *interprétation* par le LLM, qui produit par exemple l'intention `create_task` et l'entité `title` = « Préparer le rapport de stage » ; (3) *validation et normalisation*, qui détectent valeurs absentes, incohérentes ou insuffisantes ; (4) *détermination de l'action*, par exemple `create_task` vers le connecteur Todo et `create_event` vers le connecteur Agenda (aucune opération externe pour une salutation ou une demande de capacités) ; (5) *proposition et confirmation* ; (6) *exécution via le connecteur*, qui traduit l'action interne en requête conforme à l'API ; (7) *traitement de la réponse*, normalisée avant retour à l'utilisateur ; (8) *gestion des erreurs* : erreurs LLM temporaires (retry, fallback) et erreurs d'API traitées au niveau des connecteurs et de la logique applicative. Cette chaîne permet de tester séparément chaque étape.

4.2 Conception du pipeline conversationnel

Le pipeline transforme progressivement une requête en une représentation structurée utilisable pour une action de productivité ou une réponse conversationnelle : normalisation → classification de l'intention → extraction des entités → validation et normalisation → construction de l'action → confirmation → exécution → réponse.

4.2.1 Réception et normalisation des requêtes

La requête est reçue puis préparée pour l'interprétation sans en altérer le sens. L'objectif n'est pas de remplacer la compréhension sémantique du modèle par des règles lexicales : la normalisation prépare l'entrée, le LLM interprète les formulations variées, y compris dans plusieurs langues. Une requête issue de la transcription vocale entre ici dans le même flux qu'une requête

saisie.

4.2.2 Détection et classification des intentions

Le LLM associe la requête à une intention parmi les catégories prises en charge (tableau 4.2). L'application détermine ensuite le traitement correspondant ; une intention inconnue ou insuffisamment déterminée est orientée vers le traitement des demandes non reconnues plutôt que vers une exécution arbitraire.

Table 4.2 – Intentions prises en charge

Catégorie	Intentions
Tâches	<code>create_task</code> , <code>modify_task</code> , <code>delete_task</code>
Événements	<code>create_event</code> , <code>modify_event</code> , <code>delete_event</code>
Synthèse	<code>summarize_period</code>
Conversation	<code>greeting</code> , <code>farewell</code> , <code>thanks</code> , <code>small_talk</code>
Capacités	<code>capabilities</code>
Hors périmètre	<code>unrecognized</code>

4.2.3 Extraction et normalisation des entités

Selon l'opération, les entités sont : titre de tâche ou d'événement, cible d'une modification ou suppression, date, heure, durée, informations d'identification de la ressource. Les valeurs extraites ne sont transmises aux API qu'après normalisation et confirmation de l'utilisateur. Dans l'implémentation, le modèle exprime la date d'un événement au format ISO 8601 attendu par le connecteur Agenda, et les durées (« 30 minutes », « 1h30 ») sont converties en minutes puis normalisées par `normalizeActionEntities`. La phase de validation distingue quatre situations : informations complètes (le traitement continue), incomplètes (demande de précision ou interruption), ambiguës (pas d'exécution tant que l'ambiguïté subsiste) et invalides (rejet ou correction).

4.2.4 Construction des actions de productivité

Intention et entités validées sont transformées en une action structurée, point de transition entre compréhension et exécution. Le LLM ne dialogue jamais avec les services Todo ou Agenda.

```
proposedAction {
  interactionId, intent, details, targetId, requiresValidation
}
```

Les actions `create_task`, `modify_task` et `delete_task` sont orientées vers le connecteur Todo, les actions sur les événements vers le connecteur Agenda. Cette représentation intermédiaire améliore la maintenabilité : un changement d'API est principalement confiné au connecteur.

4.2.5 Gestion des réponses conversationnelles

Pour une intention conversationnelle (`greeting`, `thanks`, `small_talk`), aucune API n'est sollicitée et la réponse est produite directement. Pour une action opérationnelle, la réponse dépend du résultat : confirmation en cas de succès, réponse contrôlée en cas d'échec. Une demande insuffisamment fiable ou hors périmètre ne déclenche aucune action externe. Les problèmes fonctionnels (information manquante ou ambiguë) sont distingués des problèmes techniques (réseau, fournisseur, API).

4.3 Conception de la stratégie de résilience

Le caractère probabiliste du LLM et la dépendance au fournisseur imposent une résilience à plusieurs niveaux, résumée au tableau 4.3.

4.3.1 Gestion des erreurs du modèle principal

Les erreurs temporaires (limitation de débit, erreur serveur, problème réseau, délai dépassé) peuvent justifier un retry borné; les erreurs indiquant un problème permanent ne sont pas réessayées. Le nombre de tentatives est limité pour éviter une boucle infinie et une charge inutile sur le fournisseur. Les erreurs non récupérables conduisent à un échec contrôlé.

4.3.2 Mécanisme de fallback

Si le modèle principal ne permet pas de poursuivre après les mécanismes de récupération, un modèle de secours est utilisé. Le fallback est un second niveau de protection, non un moyen de masquer toutes les erreurs : une erreur 429 peut conduire directement au basculement, une erreur serveur temporaire provoque d'abord un retry contrôlé, une erreur non récupérable n'est pas répétée. Si le secours échoue aussi, une erreur contrôlée est retournée (figure 4.2).

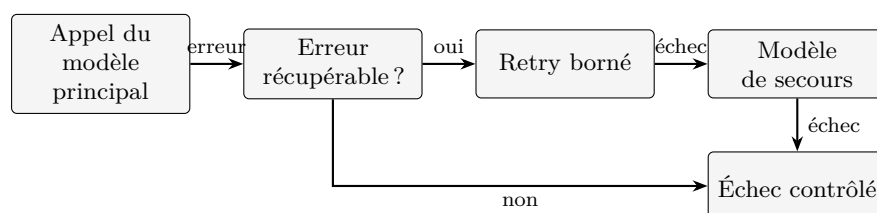


Figure 4.2 – Logique de résilience face aux erreurs du fournisseur LLM (un succès à toute étape reprend le pipeline)

4.3.3 Routage déterministe des actions

La résilience repose aussi sur la réduction de la dépendance au LLM lorsque le contexte applicatif suffit. Une fois l'intention validée, chaque type d'action est explicitement associé à son connecteur (`create_task`, `modify_task`, `delete_task` vers `Todo`; `create_event`, `modify_event`, `delete_event` vers `Agenda`), sans nouvelle sollicitation du modèle. Le comportement est plus prévisible et plus facile à tester : le LLM comprend, la logique déterministe valide et route, les connecteurs exécutent.

4.3.4 Gestion des cas non reconnus

Une requête est non reconnue si elle ne correspond à aucune intention prise en charge, est trop ambiguë ou manque d'informations obligatoires. L'intention est alors **unrecognized**, aucun connecteur n'est appelé et une réponse conversationnelle est retournée. Cette règle est essentielle pour la modification et la suppression : une cible insuffisamment identifiée ne doit pas conduire à une modification arbitraire.

Table 4.3 – Niveaux de la stratégie de résilience

Niveau	Mécanisme	Objectif
1	Classification des erreurs	Distinguer erreurs récupérables et non récupérables
2	Retry limité	Récupérer certaines erreurs temporaires
3	Fallback	Maintenir le traitement si le modèle principal reste indisponible
4	Routage déterministe	Réduire la dépendance au LLM pour les décisions applicatives
5	Cas non reconnus	Éviter d’agir sur une interprétation incertaine
6	Tests automatisés	Vérifier le comportement attendu de chaque mécanisme

4.4 Conception des connecteurs API

4.4.1 Architecture des connecteurs

Les connecteurs forment une couche d’abstraction entre l’orchestrateur et les API externes : orchestrateur $\rightarrow$ contrat du connecteur $\rightarrow$ connecteur Todo ou Agenda $\rightarrow$ requête HTTP $\rightarrow$ API. L’orchestrateur ignore les détails du service ; le connecteur se charge de construire la requête adaptée. Des implémentations de test peuvent remplacer les connecteurs réels si elles respectent le contrat, ce qui permet de tester l’orchestration sans service externe.

4.4.2 Contrats d’interface Todo et Agenda

Le contrat Todo regroupe `createTask`, `modifyTask` et `deleteTask` ; le contrat Agenda `createEvent`, `modifyEvent` et `deleteEvent`. L’orchestrateur n’a pas à connaître l’URL, la méthode HTTP, le corps de la requête, les en-têtes, le format de réponse ni le traitement des erreurs HTTP. Les contrats permettent de tester séparément la logique applicative (quelle opération ?) et l’intégration API (traduction correcte en HTTP).

4.4.3 Gestion des paramètres, des réponses et des erreurs HTTP

Le connecteur sélectionne l’endpoint et la méthode, prépare paramètres et corps, ajoute les informations d’authentification fournies par la configuration d’environnement, puis interprète la réponse en résultat normalisé. Les données temporelles sont transmises dans le format attendu par le service Agenda.

Table 4.4 – Traitement général des réponses HTTP par les connecteurs

Catégorie	Exemple	Traitement général
Succès	2xx	Convertir et retourner le résultat
Erreur client	4xx (authentification, ressource inexistante)	Identifier le problème, retourner une erreur contrôlée, ne pas répéter une requête identique
Erreur serveur	5xx	Considérer une indisponibilité éventuellement temporaire
Erreur réseau	Timeout, connexion interrompue	Traiter comme erreur technique

Les connecteurs retournent des erreurs suffisamment structurées pour que la couche applicative réponde à l’utilisateur sans exposer inutilement les détails internes du service.

4.4.4 Séparation entre logique conversationnelle et intégration externe

La logique conversationnelle répond à « que demande l’utilisateur, quelle intention, quelles informations, l’action est-elle valide ? » ; les connecteurs répondent aux questions techniques

(endpoint, méthode, paramètres, en-têtes, interprétation de la réponse, erreurs HTTP). Cette faible dépendance permet d'ajouter une intention sans modifier la couche HTTP, de confiner un changement d'API au connecteur correspondant, de tester chaque côté indépendamment et d'ajouter un service par un nouveau connecteur respectant les conventions de l'application. Le choix entre connecteurs réels et connecteurs simulés (*stub*) est fait par configuration.

4.5 Conception de la persistance

La base de données stocke les données propres à l'application et l'historique des interactions ; les tâches et événements restent gérés par les services Todo et Agenda via leurs API, ce qui évite deux sources de vérité concurrentes. L'accès aux données passe par Prisma, qui définit le modèle de façon déclarative, génère le client typé et gère les migrations. L'environnement de développement et de validation utilise SQLite, léger et reproductible ; un déploiement à grande échelle demanderait une réflexion complémentaire (section 5.10.3).

Une interaction regroupe la requête de l'utilisateur, le résultat de l'interprétation, l'action éventuelle (proposée, confirmée, exécutée ou échouée) et la réponse de l'assistant, avec des informations temporelles pour restituer l'historique chronologiquement. Le modèle distingue les interactions conversationnelles pures de celles ayant conduit à une opération externe, ainsi que les demandes non reconnues. Le LLM n'accède pas à la base : le backend contrôle les lectures et écritures, ce qui facilite aussi le test de la persistance indépendamment du modèle.

4.6 Conception de la stratégie d'évaluation

L'évaluation s'organise en niveaux complémentaires pour distinguer une régression réelle d'un problème temporaire du fournisseur : jeux de données structurés (intentions, entités), cas de généralisation, tests déterministes et contrôle de régression.

4.6.1 Organisation des jeux de données de test

- **Jeu de données fonctionnel** : requêtes représentatives de toutes les intentions (tableau 4.2), chacune associée à une intention attendue et, si besoin, à des entités attendues.
- **Tests de généralisation** : formulations différentes de celles de référence pour les mêmes intentions, afin de vérifier que le système ne se limite pas à quelques formulations.
- **Tests d'extraction des entités** : titre de tâche, cible d'une modification, nom d'événement, informations temporelles.
- **Tests déterministes** : contrats des connecteurs, appels aux API Todo et Agenda, routage, traitement des erreurs, retry et fallback, validation contractuelle de la transcription vocale, compilation. Ils ne dépendent pas du comportement probabiliste du modèle et évitent d'attribuer au LLM des erreurs d'un composant déterministe.

4.6.2 Définition des métriques

L'*exactitude des intentions* est la proportion d'intentions prédites correctes :

$$\text{Accuracy} = \frac{N_{\text{prédictions correctes}}}{N_{\text{prédictions totales}}} \quad (4.1)$$

Le *rappel* et la *précision* des entités s’expriment à partir des vrais positifs (TP), faux négatifs (FN, entités attendues non extraites) et faux positifs (FP, entités extraites non attendues) :

$$\text{Recall} = \frac{TP}{TP + FN} \quad \text{Precision} = \frac{TP}{TP + FP} \quad (4.2)$$

Une extraction supplémentaire non pertinente est problématique même lorsque les entités attendues sont retrouvées : rappel et précision sont donc analysés conjointement, complétés par l’exactitude des valeurs extraites. Le tableau 4.5 résume l’ensemble des métriques ; les erreurs du fournisseur sont comptabilisées à part pour ne pas être confondues avec des erreurs de classification.

Table 4.5 – Métriques d’évaluation

Métrique	Objectif
Intent accuracy	Correction de la classification des intentions
Entity recall	Capacité à extraire les entités attendues
Entity precision	Pertinence des entités extraites (complément : taux d’entités inattendues)
Entity value correctness	Correction des valeurs extraites
Coverage	Proportion de cas effectivement évalués
Provider error rate	Cas interrompus par une erreur du fournisseur
Latence moyenne	Temps de traitement observé ; indicateur et non mesure de qualité fonctionnelle
Confiance moyenne	Indicateur de stabilité des prédictions, ne remplaçant pas les mesures d’exactitude

4.6.3 Seuils d’acceptation et détection des régressions

Le projet définit des valeurs de référence dans un fichier de baseline ; elles servent de seuils de contrôle et **ne constituent pas des résultats expérimentaux**.

Table 4.6 – Seuils de référence du regression gate

Critère	Valeur de référence
Baseline de généralisation des intentions	100 %
Baseline d’extraction des entités	100 %
Exactitude minimale attendue	95 %
Régression maximale autorisée	5 points de pourcentage

La régression se calcule par différence entre la valeur de référence et la valeur courante : avec une référence de 100 % et une valeur courante de 97 %, la diminution est de 3 points, inférieure aux 5 points autorisés. Une valeur sous le seuil minimal ou une diminution supérieure à la marge entraîne l’échec du contrôle. Cette règle est appliquée sur des métriques distinctes de l’évaluation descriptive du jeu complet, qui fournit l’analyse détaillée.

Une évaluation partiellement interrompue par une réponse 429 ne traduit pas nécessairement une régression du code : erreurs du fournisseur, couverture et métriques sur les cas effectivement traités sont conservées séparément. Cette stratégie est ensuite intégrée à la CI.

Conclusion

La conception sépare l’interprétation probabiliste du LLM de l’exécution déterministe : pipeline conversationnel, résilience à plusieurs niveaux, connecteurs encapsulant les API, persistance

distincte des services externes, et stratégie d'évaluation distinguant erreurs applicatives et erreurs du fournisseur. Le chapitre suivant présente la réalisation de ces composants et les résultats obtenus.

Chapitre 5

Réalisation, tests et validation

Introduction

Ce chapitre décrit la mise en œuvre des choix de conception : environnement de développement, pipeline conversationnel, interface utilisateur, interaction vocale, mécanismes de résilience, connecteurs de productivité, puis tests fonctionnels, évaluation quantitative, intégration continue et limites. Les résultats des différents niveaux d'évaluation (évaluation locale du jeu de données, généralisation, exécution CI, regression gate) sont présentés séparément et ne doivent pas être confondus.

5.1 Environnement de développement

5.1.1 Environnement matériel et logiciel

Le développement a été réalisé sur un poste personnel sous Windows, permettant d'exécuter localement le backend, le frontend et les scripts de test. Les clés d'accès aux fournisseurs ne figurent pas dans le code source : elles sont gérées par la configuration de l'environnement.

5.1.2 Technologies et bibliothèques utilisées

Table 5.1 – Environnement et technologies utilisés

Élément	Rôle dans le projet
Visual Studio Code, Git, GitHub	Édition du code, gestion des versions, hébergement du dépôt
Node.js, npm, TypeScript	Exécution du backend, gestion des dépendances, développement typé (structures de données explicites entre les étapes du traitement)
Prisma, SQLite	Modélisation, client et migrations ; persistance locale de développement et de validation
API REST / HTTP	Communication avec les services de productivité
LLM via Groq	Compréhension des requêtes, classification des intentions, extraction des entités (évaluations et inférence)
Whisper via Groq	Transcription de l'entrée vocale
Expo / React Native	Interface frontend ; adresse du backend configurable par variable d'environnement
Expo Speech	Synthèse vocale des réponses de l'assistant
ESLint, TypeScript (frontend)	Analyse statique et typage de l'interface
GitHub Actions	Automatisation des tests et des contrôles de validation

5.1.3 Organisation du dépôt et gestion des versions

Le dépôt contient un backend (interprétation, intentions et entités, orchestration, résilience, connecteurs, scripts d'évaluation), un frontend Expo / React Native et la configuration GitHub Actions. Le dépôt du projet est disponible sur GitHub : github.com/zgarnirawen/Multilingual-AI-Productivity-Assistant.

```
Multilingual-AI-Productivity-Assistant/  
  backend/ src/ (services, connectors, ...) prisma/ evaluation-results/  
  frontend/ src/app/ .env.example  
  .github/workflows/assistant-validation.yml  
  README.md
```

Le développement s’est appuyé sur des branches dédiées, validées avant intégration à la branche principale :

- `feature/real-agenda-todo-api` : connecteurs réels de productivité ;
- `feature/resilient-intent-inference` : résilience de l’inférence ;
- `fix/task-target-query-normalization` : normalisation des requêtes ;
- `feat/evaluation-dataset` : évaluation automatisée et validation frontend, proposée à l’intégration par pull request.

La gestion des versions participe ainsi à la démarche de validation en associant chaque évolution à ses vérifications.

5.2 Réalisation du pipeline conversationnel

Le pipeline transforme une requête en représentation structurée, transmise si nécessaire au connecteur approprié : traitement de la requête → classification → extraction → normalisation et validation → construction de l’action → proposition et confirmation → connecteur Todo ou Agenda.

5.2.1 Traitement des requêtes

Le backend transmet la requête au composant d’interprétation, qui détermine l’intention et identifie les éléments importants. Les formulations « Créer une tâche pour préparer le rapport », « Ajoute-moi une tâche : préparer le rapport » ou « Je dois préparer le rapport, crée une tâche pour ça » doivent toutes être interprétées comme une création de tâche. Le résultat est une représentation structurée et non une réponse textuelle brute, car les composants d’exécution exigent des données cohérentes. Les intentions conversationnelles (`greeting`, `farewell`, `thanks`, `small_talk`, `capabilities`) sont traitées séparément des intentions d’action ; une requête non interprétable reçoit `unrecognized` et ne déclenche aucune opération externe.

5.2.2 Classification des intentions

La classification est réalisée par le LLM, dont le résultat est contrôlé par la logique applicative avant tout usage. « Crée une tâche pour préparer la présentation » doit donner `create_task` ; « Organise une réunion avec Sara vendredi à 14 heures » doit donner `create_event`. Le routage déterministe (section 5.5.3) en découle. Des tests de généralisation sur des formulations inédites vérifient que la classification ne dépend pas des formulations du jeu initial (section 5.7.4).

5.2.3 Extraction des entités

Pour « Créer une réunion avec Sara vendredi à 14h pendant une heure », la représentation attendue comprend l’intention `create_event` et les entités participant (Sara), date (vendredi), heure (14h) et durée (1 heure). Les données sont ensuite contrôlées : « Crée une réunion avec Sara » permet d’identifier l’intention et le participant mais pas tous les paramètres d’un événement, et le système ne doit pas considérer une information manquante comme une valeur valide

générée arbitrairement. L'évaluation montre que l'extraction est plus sensible que la classification (section 5.8).

5.2.4 Normalisation et résolution des paramètres d'action

La sortie du LLM n'est pas utilisée directement pour appeler une API : les entités sont validées et normalisées, puis les cibles sont résolues, pour produire des paramètres compatibles avec les contrats des connecteurs. Le prompt de classification demande au modèle d'exprimer la date d'un événement au format ISO 8601 et la durée directement en minutes (« 30 minutes » donne 30, « 1h30 » donne 90) ; la fonction `normalizeActionEntities` valide et normalise ensuite notamment les durées, les noms de contacts et les requêtes de ciblage. La résolution de cible identifie la ressource visée par une modification ou suppression. Par exemple, « Modifie la tâche rapport final et renomme-la en rapport finalisé » donne `modify_task` avec la cible « rapport final » et la nouvelle valeur « rapport finalisé ». Une requête incomplète ou ambiguë n'est pas exécutée. Cette étape matérialise la frontière entre interprétation probabiliste et exécution déterministe.

5.3 Interface utilisateur

L'interface est une application Expo / React Native dont l'écran principal est une conversation. L'utilisateur y saisit un message ou dicte sa requête ; pour les intentions de création, l'assistant affiche une proposition d'action (composant `ActionProposal`) accompagnée des boutons *Valider*, *Modifier* et *Ignorer*. L'action n'est transmise au backend (POST `/assistant/confirm-action`) qu'après validation par l'utilisateur.

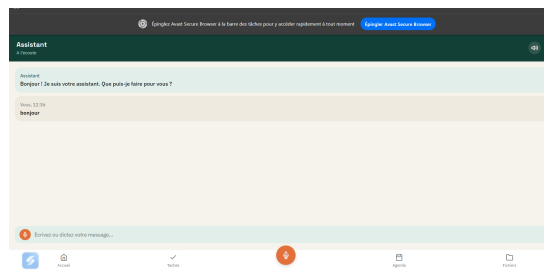


Figure 5.1 – Écran de conversation de l'assistant

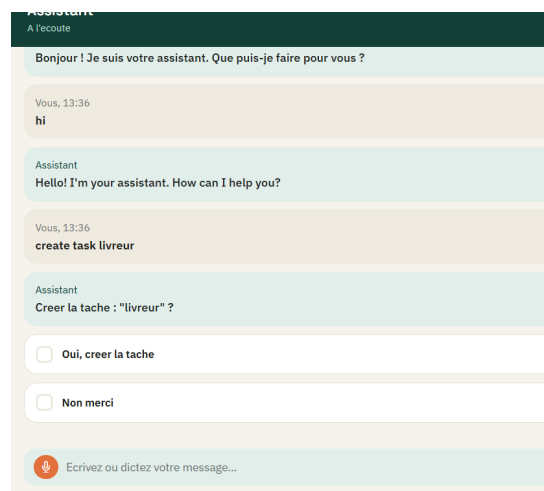


Figure 5.2 – Proposition d'action soumise à confirmation

5.4 Réalisation de l'interaction vocale

L'interaction vocale ajoute à l'assistant une modalité d'entrée (dictée) et de sortie (lecture des réponses) sans modifier le cœur du pipeline. La figure 5.3 résume le flux.

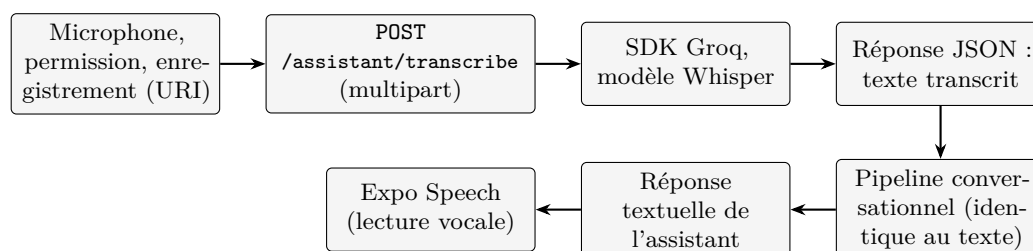


Figure 5.3 – Flux de l'interaction vocale : entrée par transcription, sortie par synthèse

5.4.1 Acquisition et traitement de l'entrée vocale

Côté frontend, l'écran de conversation propose, en plus de la saisie texte, un contrôle d'enregistrement par le microphone de l'appareil. Avant le premier enregistrement, l'application demande à l'utilisateur la permission d'accéder au microphone ; sans cette permission, aucune capture n'est possible. L'enregistrement audio est piloté par un état d'enregistrement de l'interface, qui reflète l'enregistrement en cours et déclenche son arrêt. À l'arrêt, l'enregistrement fournit un fichier audio désigné par un URI local. Ce fichier est ensuite préparé pour l'envoi au backend : il est ajouté comme partie de fichier d'une requête `multipart/form-data`.

5.4.2 Conversion Speech-to-Text

Le frontend envoie l'audio au backend par une requête `POST` sur l'endpoint `/assistant/transcribe`, au format `multipart/form-data`, avec le fichier audio en pièce jointe. Le backend vérifie la présence du fichier : en son absence, il retourne une réponse d'erreur au lieu de poursuivre le traitement. Il transmet ensuite l'audio au service de transcription Groq par son SDK, en utilisant un modèle de transcription Whisper (`whisper-large-v3`). Lorsque la langue est explicitement définie, elle est transmise comme paramètre de configuration de la transcription ; sinon, le service applique son comportement par défaut. La réponse est un objet JSON contenant le texte transcrit. Les erreurs (échec du service de transcription ou autre exception) sont interceptées et retournées sous forme de réponse d'erreur ; côté interface, un échec de transcription est affiché dans la conversation sous la forme d'une bulle d'erreur de l'assistant.

Table 5.2 – Contrat de l'endpoint de transcription

Aspect	Comportement
Méthode et route	<code>POST /assistant/transcribe</code>
Format de requête	<code>multipart/form-data</code> avec un fichier audio
Validation	Absence de fichier audio : réponse d'erreur
Service appelé	SDK Groq, modèle de transcription Whisper
Langue	Paramètre de langue transmis lorsqu'il est explicitement défini
Réponse nominale	JSON contenant le texte transcrit
Erreurs	Erreurs du service ou du traitement interceptées et renvoyées sous forme de réponse d'erreur

5.4.3 Intégration du texte transcrit dans le pipeline conversationnel

La voix est une modalité d'entrée supplémentaire et non une architecture de raisonnement distincte : le texte transcrit est injecté dans le même pipeline que les messages saisis. Après transcription, il suit les mêmes étapes d'interprétation, d'extraction d'entités, de validation, de routage et d'exécution, avec la même confirmation avant toute action. Le mode d'entrée (vocal ou texte) est transmis à l'envoi du message (`inputMode`) et conservé dans l'historique des interactions, ce qui permet de distinguer l'origine d'une requête sans dupliquer la logique de traitement.

5.4.4 Synthèse vocale des réponses

La restitution vocale utilise Expo Speech côté frontend : à partir de la réponse textuelle de l'assistant, l'application déclenche une lecture à voix haute. La langue configurée dans le code est le français (`fr-FR`). Avant chaque nouvelle lecture, la lecture en cours est arrêtée pour éviter le chevauchement de deux voix. Un contrôle d'activation et de désactivation (muet) est disponible dans l'en-tête de l'interface. Seules les réponses de l'assistant sont lues, et non le texte transcrit de l'utilisateur ; la réponse textuelle reste affichée dans tous les cas.

5.4.5 Validation de l'intégration vocale

Le projet contient une validation *contractuelle* de l'intégration Speech-to-Text, exécutée dans le pipeline CI (job de validation hors ligne, section 5.9.1). Elle vérifie les points listés au tableau 5.3.

Table 5.3 – Points vérifiés par la validation contractuelle du Speech-to-Text

Point vérifié	Nature de la vérification
Existence de l'endpoint	La route de transcription est bien exposée par le backend
Réception multipart	L'endpoint reçoit une requête <code>multipart/form-data</code>
Absence de fichier	Le cas sans fichier audio est validé (réponse d'erreur)
SDK Groq et modèle Whisper	L'intégration utilise le SDK Groq et un modèle Whisper
Configuration de la langue	La configuration de la langue est prise en compte
Format de réponse	La réponse JSON contient le texte transcrit
Gestion des erreurs	Les erreurs sont traitées

Cette validation contrôle la conformité du contrat d'intégration. Elle **ne constitue pas un benchmark de précision** de la transcription : aucun taux d'erreur sur les mots (WER) ni aucune mesure de précision STT n'a été produit dans ce projet.

5.4.6 Limites de l'interaction vocale

- dépendance au service de transcription externe (disponibilité et qualité) ;
- absence de benchmark audio complet ;
- validation CI principalement contractuelle ;
- absence de test automatisé de bout en bout microphone → STT → assistant → TTS ;
- contraintes éventuelles liées aux environnements d'exécution (permissions et capacités audio de l'appareil ou de l'émulateur).

5.5 Réalisation des mécanismes de résilience

5.5.1 Gestion des erreurs des fournisseurs LLM

La classification des erreurs détermine le comportement (tableau 5.4). Les erreurs 429 (limitation temporaire du débit) donnent lieu à une nouvelle tentative puis, si besoin, au modèle de secours ; les erreurs 5xx comme 503 provoquent un nombre limité de tentatives pour éviter boucle infinie et hausse inutile de la latence ; les timeouts réseau sont traités comme récupérables ; une erreur 401 n'est pas répétée, car la même requête ne résoudrait pas un problème d'authentification, et elle est traitée comme non récupérable.

Table 5.4 – Comportements implémentés selon le type d'erreur LLM

Scénario	Comportement attendu
429, limitation de débit	Basculement vers le modèle de secours (retry avant fallback si nécessaire)
503 temporaire	Retry du fournisseur principal
Plusieurs 503 successifs	Retry borné puis fallback
Timeout réseau (ETIMEDOUT)	Nouvelle tentative
401, authentification	Pas de retry automatique
Erreur applicative non récupérable	Arrêt contrôlé
Échec du principal et du secours	Échec contrôlé

5.5.2 Implémentation du fallback

Le fallback intervient uniquement lorsque les tentatives auprès du fournisseur principal sont épuisées et que l'erreur est compatible avec une reprise ; le fournisseur principal reste le chemin nominal. Il a été validé par des tests automatisés couvrant les scénarios du tableau 5.4 : basculement sur 429, retry borné sur 5xx, reprise après timeout, absence de retry sur 401 et sur erreur non récupérable, et épuisement du fallback (principal et secours en échec). Le fallback assure la continuité de l'étape d'interprétation mais ne remplace pas les contrôles applicatifs effectués sur le résultat du modèle.

5.5.3 Routage déterministe des actions

Le routage ne dépend pas d'une nouvelle décision du modèle : le backend associe les trois intentions sur les tâches au Todo Connector et les trois intentions sur les événements à l'Agenda Connector, les autres intentions relevant du traitement conversationnel. Le modèle ne choisit donc jamais le service externe ; le comportement est plus prévisible, testable (tests de routage) et extensible par l'ajout d'un connecteur.

5.5.4 Gestion des réponses non reconnues

`unrecognized` représente une demande hors périmètre, ambiguë, correspondant à aucune intention définie ou dont les informations sont insuffisantes. Aucune action n'est alors devinée ni exécutée. Cette situation, d'ordre fonctionnel, se distingue d'une erreur technique du fournisseur ; une intention reconnue mais dépourvue de paramètres suffisants passe d'abord par la validation avant tout envoi au service externe.

5.6 Intégration des connecteurs de productivité

Les connecteurs forment la frontière entre le backend et les API : ils encapsulent la construction des requêtes HTTP, la transmission des paramètres et de l'authentification, l'interprétation des réponses et le traitement des erreurs (figure 4.1).

5.6.1 Connecteur Todo

Le Todo Connector prend en charge la création, la modification et la suppression de tâches (`createTask`, `modifyTask`, `deleteTask`). Il reçoit des paramètres déjà interprétés et validés et les convertit au format attendu par l'API ; la création de tâche aboutit, côté connecteur HTTP, à une requête `POST /tasks`. Pour « Crée une tâche pour préparer le rapport », l'intention `create_task` est validée, puis l'action est transmise au connecteur, qui construit la requête de création. La logique conversationnelle ne contient aucun détail d'endpoint ou de format HTTP. Les modifications et suppressions requièrent une cible permettant d'identifier la ressource.

5.6.2 Connecteur Agenda

L'Agenda Connector assure de même la création, la modification et la suppression d'événements (`createEvent`, `modifyEvent`, `deleteEvent`) à partir des informations extraites et normalisées (titre, date, heure, durée, paramètres de l'opération) ; la création d'événement aboutit à une requête `POST /events`. Des connecteurs simulés (*stub*) sont conservés pour des tests indépendants des services externes, le choix entre connecteur réel et simulé étant fait par configuration, tandis que les connecteurs réels valident l'intégration avec les API.

5.6.3 Requêtes HTTP, réponses et authentification

Une requête HTTP comprend une méthode, une URL, des en-têtes, l'authentification et un corps. Le connecteur analyse la réponse pour déterminer succès ou erreur et ne transmet pas d'objet HTTP brut à la logique conversationnelle. Les situations considérées sont : succès (conversion du résultat), ressource inexistante (404, signalement de l'absence), erreur serveur (5xx, erreur d'intégration), erreur d'authentification, erreur réseau et réponse inattendue (gestion contrôlée).

L'authentification est gérée dans la couche des connecteurs : les informations sensibles restent dans la configuration de l'environnement et sont ajoutées aux en-têtes lors de la construction de la requête. Le backend n'a pas à connaître les détails d'authentification de chaque service, les secrets ne sont pas dans le code source et la configuration peut changer sans modifier la logique conversationnelle. Cette authentification d'accès aux services est distincte de l'authentification globale de l'application, hors périmètre.

5.6.4 Validation des contrats d'intégration

Deux catégories de tests vérifient les connecteurs : les tests de *contrats*, qui confirment que les opérations exposées respectent les comportements attendus et que les données restent cohérentes entre orchestrateur et couche d'intégration, et les tests d'*intégration API*, qui couvrent les opérations Todo et Agenda, la transmission de l'authentification, les ressources inexistantes, les erreurs serveur et la gestion contrôlée des réponses en erreur. Les scénarios validés comprennent les cas de succès, les réponses 404, les erreurs 500 et les problèmes d'authentification. Des scripts automatisés dédiés ont été exécutés avec succès lors de la validation du projet.

5.7 Tests et validation fonctionnelle

5.7.1 Stratégie de test

Table 5.5 – Catégories de tests

Catégorie	Objectif principal
Compilation	Vérifier que le projet se construit correctement
Connecteurs et API	Vérifier les contrats et l'intégration avec les API
Entités	Vérifier l'extraction des paramètres utiles aux actions
Généralisation	Vérifier la reconnaissance d'intentions sur des formulations inédites
Fallback	Vérifier la résistance aux erreurs des fournisseurs LLM
Routage et titres	Vérifier l'association intention/connecteur et le routage des titres
Transcription vocale	Vérifier le contrat d'intégration STT (section 5.4)
Scénarios conversationnels	Vérifier le comportement global du pipeline
Évaluation par jeu de données	Mesurer les performances globales (section 5.8)
Frontend	Vérifier le typage et la qualité statique de l'interface

Une partie importante des tests est déterministe et exécutable systématiquement ; les tests utilisant un fournisseur LLM sont traités séparément en raison des possibles erreurs de disponibilité ou de limitation de débit. Cette séparation évite de considérer une indisponibilité du fournisseur comme une régression du code (section 5.9.1).

5.7.2 Tests des connecteurs

Les tests des connecteurs ont couvert la création, la modification et la suppression de tâches et d'événements, la gestion de l'authentification, d'une ressource inexistante, d'une erreur serveur et des erreurs d'intégration. Les validations automatisées des contrats et des API ont été exécutées avec succès (succès, authentification, 404, 500).

5.7.3 Tests d'extraction des entités

Une intention correctement identifiée peut s'accompagner d'une extraction erronée ; des tests ciblés couvrent donc titres de tâches et d'événements, personnes, dates, heures, durées et cibles de modification ou suppression. Ils vérifient par exemple « Appelle Karim demain » ou « Crée une réunion avec Sara vendredi à 14h pendant une heure ». Les 5 cas des scénarios d'extraction ciblés ont été correctement traités. L'évaluation sur le jeu complet (section 5.8) montre que l'extraction reste plus sensible aux variations que la classification, ce qui justifie des métriques dédiées (rappel, précision, correction des valeurs).

5.7.4 Tests de généralisation des intentions

Ces tests vérifient que l'assistant ne se limite pas aux formulations exactes de sa conception : par exemple, « Crée une tâche pour préparer le rapport », « Ajoute une tâche : préparer le rapport » et « Je dois préparer le rapport, ajoute-le à mes tâches » doivent toutes donner `create_task`. Le jeu comporte 33 cas couvrant les intentions fonctionnelles et conversationnelles ; **33 cas sur 33** ont été correctement classés (100 %). Ce résultat est distinct de l'évaluation du jeu de 65 cas et ne garantit pas la reconnaissance de toutes les formulations possibles ; ces tests constituent surtout un mécanisme de détection de régression (modification du prompt, de l'interprétation ou du routage).

5.7.5 Tests des mécanismes de fallback

Les scénarios simulés sont ceux du tableau 5.4. La validation automatisée a confirmé le failover sur 429, le retry borné sur 5xx, le retry réseau, l'absence de retry pour les erreurs non récupérables et la gestion de l'épuisement du fallback. Ce test est important, la disponibilité du fournisseur étant une dépendance externe : l'application doit distinguer une indisponibilité temporaire d'un problème de configuration.

5.7.6 Tests des scénarios conversationnels

Ils vérifient l'enchaînement classification, extraction, validation, construction de l'action, routage, connecteur et résultat sur des scénarios représentatifs.

Table 5.6 – Scénarios conversationnels de référence

Scénario	Intention	Vérification principale
Création de tâche	<code>create_task</code>	Extraction du titre, routage Todo
Modification de tâche	<code>modify_task</code>	Identification de la cible et nouvelle valeur
Suppression de tâche	<code>delete_task</code>	Résolution de la cible
Création d'événement	<code>create_event</code>	Gestion des paramètres temporels
Modification d'événement	<code>modify_event</code>	Identification et modification de la cible
Suppression d'événement	<code>delete_event</code>	Routage vers Agenda
Salutation, remerciement	<code>greeting, thanks</code>	Réponse conversationnelle sans action externe
Demande de capacités	<code>capabilities</code>	Présentation des fonctionnalités supportées
Demande non reconnue	<code>unrecognized</code>	Absence d'action externe

5.8 Évaluation quantitative du système

L'évaluation quantitative complète les tests fonctionnels par des indicateurs numériques. Elle repose sur un jeu de 65 cas couvrant les intentions prises en charge ; l'évaluation locale est analysée ici, l'exécution CI en section 5.9.4, car l'évaluation dépend d'un service LLM externe susceptible d'erreurs de disponibilité ou de limitation de débit.

5.8.1 Évaluation sur le jeu de données de référence

Le système définit treize intentions dans son classifieur, couvrant les opérations de tâches et d'événements, la synthèse de période, les échanges conversationnels, la présentation des capacités et les demandes non reconnues. Le jeu de 65 cas constitue un sous-ensemble d'évaluation de ces comportements, avec plusieurs formulations pour les scénarios retenus. Les métriques sont définies au tableau 4.5. Lors de l'exécution locale, les 65 cas ont été traités sans erreur du fournisseur.

Ces résultats correspondent à une exécution donnée, non à une garantie de comportement sur toutes les requêtes possibles.

5.8.2 Classification des intentions

Sur le jeu local, l'exactitude est de $65/65 = 100\%$: aucune erreur de classification n'a été observée. Cette stabilité est importante, l'intention conditionnant le routage déterministe vers le connecteur Todo ou Agenda. Elle ne signifie pas que la classification est définitivement résolue : le dataset reste limité face à la diversité du langage naturel.

Table 5.7 – Évaluation locale du jeu de données (65 cas)

Métrique	Résultat
Nombre de cas	65
Couverture	100 %
Erreurs fournisseur	0
Exactitude des intentions	100 %
Rappel / exactitude des entités	94,4 %
Précision des entités	57,6 %
Taux d'entités inattendues	40,7 %
Latence moyenne	environ 4 106 ms
Confiance moyenne	environ 0,965

5.8.3 Rappel et précision de l'extraction des entités

Le rappel/exactitude de 94,4 % indique que le système retrouve la plupart des informations attendues ; la précision de 57,6 % et le taux d'entités inattendues de 40,7 % montrent que le modèle produit aussi des informations supplémentaires par rapport à la référence. Une extraction trop restrictive perdrait un paramètre nécessaire, une extraction trop large introduit des paramètres inutiles ou incorrects. L'extraction constitue donc, dans l'état actuel, un axe d'amélioration plus important que la classification.

5.8.4 Analyse des erreurs et des cas limites

- **Variations linguistiques** : les 33 formulations inédites ont été correctement classées, dans les limites de ce jeu.
- **Informations incomplètes ou ambiguës** : « Crée une réunion avec Sara » identifie `create_event` sans date ni heure ; le système ne doit pas compléter arbitrairement, et la validation des paramètres intervient avant l'envoi au connecteur.
- **Entités supplémentaires** : le taux de 40,7 % explique en grande partie l'écart entre rappel et précision. Il ne signifie pas que toutes les actions produites sont incorrectes, mais que la sortie contient plus d'informations que la référence.
- **Erreurs du fournisseur** : elles perturbent les exécutions en environnement CI (section 5.9.4) et sont suivies séparément.

5.8.5 Résultats et interprétation

La classification des intentions est actuellement le composant le plus stable (100 % sur le jeu local, généralisation 33/33 présentée en section 5.7.4). L'extraction des entités est plus délicate : un travail sur le contrôle et la structuration des sorties du modèle est pertinent. La latence moyenne locale (environ 4,1 s) dépend du fournisseur, du réseau et de l'environnement. Les résultats de la section 5.9.4 sont à lire séparément.

5.9 Intégration continue et contrôle qualité

Un processus d'intégration continue a été mis en place avec GitHub Actions pour exécuter systématiquement les contrôles : compilation, tests des connecteurs, routage, résilience, validation du contrat de transcription vocale, évaluation du comportement du modèle et validation du frontend. Le workflow distingue les validations déterministes des évaluations dépendant d'un fournisseur LLM externe.

5.9.1 Automatisation des tests avec GitHub Actions

La configuration est définie dans `.github/workflows/assistant-validation.yml`, organisée en trois jobs (tableau 5.8, annexe E).

Table 5.8 – Jobs et validations du workflow GitHub Actions

Job	Validations
<code>offline-validation</code>	Récupération du dépôt ; installation des dépendances ; génération du client Prisma ; build du backend ; tests déterministes : intent regression, title routing, model fallback, connector contracts, API connector tests, validation du contrat Speech-to-Text
<code>live-llm-evaluation</code>	Exécuté avec la clé du fournisseur disponible : généralisation des intentions ; extraction des entités ; évaluation du jeu de données figé (frozen dataset) ; regression gate
<code>frontend-validation</code>	Installation des dépendances (<code>npm ci</code>) ; compilation TypeScript (<code>tsc</code> , sans émission) ; analyse ESLint

Cette organisation permet de reproduire les validations sans exécution manuelle et d'éviter d'exposer des secrets dans le dépôt, la clé du fournisseur n'étant utilisée que par l'évaluation LLM.

5.9.2 Exécution des validations dans le pipeline CI

La partie déterministe constitue le premier niveau de contrôle : elle détecte les erreurs de compilation et les régressions sur les fonctionnalités déjà validées. Côté frontend, la validation CI a confirmé que `npm ci` s'exécute correctement, que `npx tsc --noEmit` ne produit pas d'erreur et qu'ESLint s'exécute ; une seule alerte, sans erreur bloquante, concerne la règle `@typescript-eslint/no-empty-object-type`.

Le projet dispose ainsi d'une **intégration continue et d'un contrôle qualité automatisé**. Il ne met pas en œuvre de déploiement continu (Continuous Deployment) vers un environnement de production.

5.9.3 Mise en place du mécanisme de régression

La référence est stockée dans `backend/evaluation-baseline.json` ; ses valeurs sont les seuils du tableau 4.6 (généralisation 100 %, extraction 100 %, exactitude minimale 95 %, régression maximale 5 points). Le gate compare les indicateurs de la nouvelle évaluation à la référence et échoue si l'exactitude passe sous le seuil minimal ou si la diminution dépasse 5 points ; sinon, il est en succès (PASS). Il est particulièrement utile pour un système à LLM, où une modification du prompt ou du traitement des entités peut changer le comportement alors que le code compile toujours. Il utilise des métriques et des critères propres, distincts de l'évaluation descriptive du jeu complet (section 5.8).

5.9.4 Analyse des résultats d'intégration continue

Lors de l'exécution analysée (workflow *AI MLOps Validation*, run #69), le statut global du workflow est réussi : validations déterministes et contrôle de régression ont été exécutés correctement. L'évaluation LLM du jeu de données a toutefois été classée **DEGRADED** : sur 65 cas planifiés, 62 ont été complétés et 3 ont rencontré des erreurs (notamment des réponses 429 de limitation de débit et une erreur d'évaluation).

Ces résultats ne doivent pas être interprétés comme une régression fonctionnelle

Table 5.9 – Évaluation LLM du run CI #69 (exécution dégradée)

Élément	Résultat CI #69
Cas planifiés / complétés	65 / 62
Couverture	95,4 %
Erreurs (fournisseur et évaluation)	3
Exactitude des intentions (cas complétés)	93,5 %
Rappel / exactitude des entités	88,2 %
Précision des entités	51,7 %
Taux d'entités inattendues	43,1 %
Latence moyenne	6 658,2 ms
Confiance moyenne	0,915
Statut de l'évaluation LLM	DEGRADED

du code. Ils sont influencés par les erreurs du fournisseur LLM (couverture réduite, latences plus élevées) et mesurés sur 62 cas au lieu de 65 ; ils ne sont pas comparables terme à terme à l'évaluation locale complète. La régression est évaluée séparément par le regression gate.

Table 5.10 – Regression gate lors de la validation CI considérée (comparaison aux seuils du tableau 4.6)

Indicateur du gate	Observé (référence)
Généralisation des intentions	100 % (100 %)
Extraction des entités (indicateur du gate)	100 % (100 %)
Régression constatée	0 point (maximum autorisé : 5)
Seuil minimal	respecté
Statut du regression gate	PASS

Le workflow permet à l'évaluation LLM de signaler une exécution dégradée sans empêcher le contrôle de régression de se poursuivre : une difficulté temporaire du service externe ne masque pas les résultats déterministes. Les résultats peuvent être conservés comme artefacts d'évaluation pour analyse ultérieure. Le tableau 5.11 synthétise les quatre niveaux de résultats, à ne pas confondre.

Table 5.11 – Les quatre niveaux de résultats d'évaluation

Niveau	Ce qu'il mesure	Résultat
Évaluation locale du jeu	Comportement descriptif sur 65 cas	Intentions 100 %, entités 94,4 % (rappel) et 57,6 % (précision)
Généralisation	Formulations inédites	33/33 (100 %)
Exécution CI #69	Évaluation LLM dégradée par le fournisseur	62/65 cas, intentions 93,5 %, statut DEGRADED
Regression gate	Écart aux seuils de référence	Pas de régression, PASS

5.10 Bilan et limites de la réalisation

5.10.1 Fonctionnalités réalisées

5.10.2 Limites de l'intégration externe

Le système dépend des fournisseurs de modèles (LLM et transcription) et des API Todo et Agenda : indisponibilité, erreur réseau, limitation de débit ou réponse inattendue peuvent affecter une requête, comme l'a montré l'exécution CI dégradée (section 5.9.4). Retry et fallback améliorent la continuité pour les erreurs temporaires mais ne suppriment pas ces dépendances, et

Table 5.12 – Fonctionnalités réalisées

Fonctionnalité	Réalisation
Compréhension et intentions	Interprétation par LLM ; intentions de tâches, d'événements et conversationnelles
Entités et normalisation	Extraction, normalisation, résolution des cibles, validation avant exécution
Tâches et événements	Création, modification, suppression via les connecteurs Todo et Agenda
Routage et confirmation	Routage déterministe, proposition d'action et confirmation
Interaction vocale	Dictée via transcription Whisper (Groq) injectée dans le même pipeline ; lecture vocale avec Expo Speech, désactivable
Résilience	Classification des erreurs, retry borné, fallback, requêtes non reconnues sans action externe
Évaluation et CI	Jeux de données, généralisation, extraction, regression gate, GitHub Actions

une évolution importante d'une API externe peut exiger l'adaptation du connecteur correspondant. La précision de l'extraction des entités (57,6 % en local) constitue une piste d'amélioration prioritaire. Les limites propres à l'interaction vocale sont détaillées en section 5.4.

5.10.3 Limites liées à l'environnement de déploiement

Le projet a été développé et validé en environnement de développement et d'intégration continue, avec SQLite ; un déploiement à grande échelle demanderait une réflexion sur la base de données, la concurrence, les sauvegardes et la supervision. Le pipeline GitHub Actions n'est pas une chaîne de déploiement continu. La latence (environ 4,1 s en local ; environ 6,7 s dans l'exécution CI dégradée) dépend du fournisseur, du réseau et de l'environnement ; un usage à plus grande échelle nécessiterait d'étudier latence, concurrence et coûts. Enfin, les secrets doivent être configurés de manière sécurisée selon l'infrastructure cible.

Conclusion

La réalisation a produit une base fonctionnelle et testée : pipeline de compréhension, entrée et sortie vocales, validation et routage, connecteurs Todo et Agenda, retry et fallback, évaluation automatisée avec contrôle de régression en CI. Les résultats doivent être lus en distinguant l'évaluation locale, la généralisation, l'exécution CI dégradée par le fournisseur et le regression gate. Disponibilité des fournisseurs, latence, précision de l'extraction des entités et validation vocale uniquement contractuelle sont les principaux axes d'amélioration.

Conclusion générale et perspectives

Ce stage avait pour objectif de concevoir et de réaliser la composante intelligente d'un assistant de productivité interprétant des requêtes en langage naturel, saisies ou dictées, pour les transformer en actions sur des tâches et des événements, avec une architecture suffisamment robuste face aux erreurs des modèles, à la variabilité des requêtes et aux dépendances externes.

L'analyse du contexte et des architectures possibles a conduit à une approche hybride : le LLM assure la compréhension, tandis que la logique applicative conserve la validation, la normalisation, le routage et l'exécution. La réalisation a livré la classification des intentions, l'extraction et la normalisation des entités, les connecteurs Todo et Agenda avec confirmation préalable, l'interaction vocale (transcription Whisper via Groq, lecture avec Expo Speech) intégrée au même pipeline, ainsi que des mécanismes de retry et de fallback.

Les résultats sont les suivants : 33 cas corrects sur 33 en généralisation ; 100 % de couverture et d'exactitude des intentions sur l'évaluation locale de 65 cas, avec un rappel des entités de 94,4 % mais une précision de 57,6 %. L'exécution CI #69, dégradée par des erreurs du fournisseur, ne traduit pas une régression du code, que le regression gate évalue séparément. L'intégration vocale n'est validée que de manière contractuelle : aucun benchmark de précision de transcription n'a été réalisé. Le projet dispose d'une intégration continue et d'un contrôle qualité automatisé, mais pas d'un déploiement continu.

Perspectives.

- améliorer l'extraction des entités pour réduire les entités parasites ;
- optimiser la latence (choix des modèles, taille des prompts, cache, réduction des appels) ;
- renforcer la gestion des fournisseurs (sélection dynamique, quotas, disponibilité) ;
- étendre les services intégrés (autres calendriers, outils de tâches ou de communication) ;
- enrichir le jeu d'évaluation (formulations multilingues, ambiguës, contextuelles) et ajouter un benchmark audio de la transcription (par exemple par WER) ainsi qu'un test automatisé de bout en bout de la chaîne vocale ;
- mettre en place supervision, sécurité des secrets et infrastructure multi-utilisateur adaptée, puis étendre le pipeline vers un déploiement continu.

Ce projet a permis de mettre en pratique l'IA, le développement backend, l'intégration d'API, les tests automatisés et l'intégration continue, et de souligner l'importance de la validation déterministe, de l'évaluation quantitative et de la résilience lorsqu'un système dépend de modèles de langage et de services externes.

Bibliographie

- [1] I. Goodfellow, Y. Bengio et A. Courville, *Deep Learning*, MIT Press, 2016.
- [2] C. D. Manning, P. Raghavan et H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.
- [3] C. Huyen, *Designing Machine Learning Systems*, O'Reilly Media, 2022.
- [4] M. Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
- [5] E. Gamma, R. Helm, R. Johnson et J. Vlissides, *Design Patterns : Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- [6] I. Sommerville, *Software Engineering*, 10^e éd., Pearson, 2015.
- [7] A. Vaswani et al., « Attention Is All You Need », *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [8] A. Radford et al., « Robust Speech Recognition via Large-Scale Weak Supervision », *International Conference on Machine Learning (ICML)*, 2023.

Webographie

- [W1] OpenAI, *OpenAI Platform Documentation* : <https://platform.openai.com/docs>
- [W2] Groq, *Groq API Documentation* (accès aux modèles de langage et de transcription) : <https://console.groq.com/docs>
- [W3] Prisma, *Prisma Documentation* : <https://www.prisma.io/docs>
- [W4] GitHub, *GitHub Actions Documentation* : <https://docs.github.com/actions>
- [W5] Node.js, *Node.js Documentation* : <https://nodejs.org/docs>
- [W6] TypeScript, *TypeScript Documentation* : <https://www.typescriptlang.org/docs>
- [W7] React Native, *React Native Documentation* : <https://reactnative.dev/docs>
- [W8] Expo, *Expo Documentation* (dont le module Speech) : <https://docs.expo.dev>
- [W9] MDN Web Docs, *HTTP* : <https://developer.mozilla.org/docs/Web/HTTP>
- [W10] Git, *Git Documentation* : <https://git-scm.com/doc>

Annexe A

Configuration des connecteurs API

Aucune clé d'API, aucun mot de passe, jeton ou secret n'est reproduit dans ce rapport : ces informations sont fournies par la configuration de l'environnement d'exécution et ajoutées aux requêtes par les connecteurs.

Table A.1 – Responsabilités et opérations des connecteurs

Connecteur	Opérations du contrat	Intentions routées
Todo Connector	<code>createTask</code> , <code>modifyTask</code> , <code>deleteTask</code>	<code>create_task</code> , <code>modify_task</code> , <code>delete_task</code>
Agenda Connector	<code>createEvent</code> , <code>modifyEvent</code> , <code>deleteEvent</code>	<code>create_event</code> , <code>modify_event</code> , <code>delete_event</code>

Table A.2 – Éléments de configuration externalisés

Élément	Gestion
Identifiants des services Todo et Agenda	Variables d'environnement, ajoutées aux en-têtes par le connecteur
Clé du fournisseur de modèles (Groq)	Configuration de l'environnement ; secret du dépôt pour l'évaluation CI
Adresse du backend (frontend)	Variable d'environnement du frontend (fichier d'exemple <code>.env.example</code>)
Choix du connecteur (réel ou simulé)	Variable de configuration <code>PRODUCTIVITY_CONNECTOR</code>

Annexe B

Exemples de requêtes et réponses HTTP

Cette annexe décrit les échanges de façon schématique ; les noms de champs exacts sont ceux du code du projet.

Table B.1 – Échange de transcription vocale

Élément	Description
Requête	POST /assistant/transcribe, en-tête Content-Type: multipart/form-data, corps : fichier audio enregistré
Réponse de succès	Objet JSON contenant le texte transcrit
Absence de fichier audio	Réponse d'erreur, aucun appel au service de transcription
Erreur du service	Erreur interceptée, réponse d'erreur affichée dans la conversation sous forme de bulle d'erreur de l'assistant

Table B.2 – Échanges de traitement d'une requête et de confirmation

Élément	Description
POST /assistant/message	Corps : <code>inputText</code> , <code>inputMode</code> ; réponse : résultat d'interprétation, proposition d'action (<code>proposedAction</code>) et message de confirmation
POST /assistant/confirm-action	Corps : <code>interactionId</code> , <code>intent</code> , <code>details</code> , <code>targetId</code> ; réponse : élément créé, interaction mise à jour et message de confirmation

Les échanges des connecteurs avec les API externes suivent le traitement du tableau 4.4 : succès (2xx) converti en résultat normalisé, erreurs d'authentification et ressource inexistante (404) retournées de façon contrôlée, erreurs serveur (500, 5xx) remontées comme erreurs d'intégration.

Annexe C

Exemples du jeu de données d'évaluation

Les requêtes ci-dessous sont les exemples utilisés dans ce rapport pour illustrer chaque catégorie d'intention ; le jeu de 65 cas complet figure dans le dépôt du projet.

Table C.1 – Exemples de requêtes par intention

Intention	Exemple de requête
<code>create_task</code>	« Crée une tâche pour préparer le rapport. » ; « Je dois préparer le rapport, ajoute-le à mes tâches. »
<code>modify_task</code>	« Modifie la tâche rapport final et renomme-la en rapport finalisé. »
<code>create_event</code>	« Créer une réunion avec Sara vendredi à 14h pendant une heure. » ; « Planifie une réunion avec Karim vendredi à 10 heures. »
Entités (extraction)	« Appelle Karim demain. »
Requête incomplète	« Crée une réunion avec Sara. » (date et heure absentes)

Annexe D

Résultats détaillés des tests automatisés

Les métriques chiffrées du jeu de données figurent aux tableaux 5.7, 5.9 et 5.10. Le tableau D.1 récapitule l'état des autres validations.

Table D.1 – Résultats des validations automatisées

Validation	Résultat
Contrats des connecteurs et tests API	Exécutés avec succès (succès, authentification, 404, 500)
Fallback et résilience	Failover sur 429, retry borné sur 5xx, retry réseau, aucun retry sur erreur non récupérable, épuisement du fallback géré
Extraction ciblée des entités	5 cas sur 5 corrects
Généralisation des intentions	33 cas sur 33 corrects
Contrat Speech-to-Text	Validation contractuelle (tableau 5.3); aucune mesure de précision de transcription
Frontend	<code>npm ci</code> et <code>tsc</code> sans erreur; ESLint avec une alerte <code>no-empty-object-type</code> , sans erreur bloquante

Annexe E

Configuration du pipeline GitHub Actions

Structure schématique du fichier `.github/workflows/assistant-validation.yml` (noms de jobs et d'étapes, sans secret) :

```
jobs:
  offline-validation: # deterministe, sans fournisseur LLM
    - checkout du depot
    - installation des dependances
    - generation du client Prisma
    - build du backend
    - intent regression / title routing / model fallback
    - connector contracts / API connector tests
    - validation du contrat Speech-to-Text
  live-llm-evaluation: # necessite la cle du fournisseur (secret du depot)
    - generalisation des intentions
    - extraction des entites
    - evaluation du jeu de donnees fige (frozen dataset)
    - regression gate (baseline : backend/evaluation-baseline.json)
  frontend-validation:
    - npm ci
    - npx tsc --noEmit
    - ESLint
```